



Sauce QA

# Playwright + Allure Reporting

A hands-on tutorial built around a **real, runnable test automation framework** for the Sauce Demo Shopify store. Every screenshot in this guide comes from the report that the project itself generates.

**What you will learn:** setup · reporter integration · steps · screenshots, video & traces · network and console capture · metadata & behaviors · retries & flaky tests · history & trends · categories · custom themes · CI/CD with GitHub Actions, Jenkins and Docker.

Playwright 1.63 · allure-playwright 3.12 · Allure Report 2.46 · TypeScript 5

# Contents

---

## 1. Introduction

- 1.1 The system under test
- 1.2 How the pieces fit together

## 2. Setup & Installation

- 2.1 Prerequisites
- 2.2 Install the project
- 2.3 Adding Allure to your own Playwright project
- 2.4 Project structure
- 2.5 First run

## 3. Basic Integration

- 3.1 Register the reporter
- 3.2 Reporter options
- 3.3 Generate, open and serve
- 3.4 Clean results between runs

## 4. Framework Architecture

- 4.1 Configuration management
- 4.2 Page Object Model
- 4.3 Fixtures: evidence for free

## 5. Allure Features Explained

- 5.1 Titles and descriptions
- 5.2 Test statuses
- 5.3 Behaviors: epic, feature, story
- 5.4 Severity
- 5.5 Tags, owner, layer, custom labels and suites
- 5.6 Detailed test steps
- 5.7 Visual evidence: screenshots, video, traces, HTML snapshots
- 5.8 Network and performance monitoring
- 5.9 API testing with Allure steps
- 5.10 Console capture and logging
- 5.11 Parameters and test data
- 5.12 Environment and executor information

- 5.13 Retries and flaky tests
- 5.14 Links and relationships
- 5.15 History and trend analysis
- 5.16 Custom attachments
- 5.17 Categories: automatic defect triage
- 5.18 Timeline and Packages

## **6. Best Practices**

- 6.1 Write reports for humans
- 6.2 Metadata discipline
- 6.3 Evidence without noise
- 6.4 Stable history
- 6.5 Flakiness
- 6.6 Speed

## **7. Troubleshooting**

## **8. Advanced Customization**

- 8.1 One configuration file
- 8.2 Custom theme and logo
- 8.3 Report name, language and filters
- 8.4 Global labels and listeners
- 8.5 Running a subset with a test plan
- 8.6 Allure Report 3

## **9. CI/CD Integration**

- 9.1 GitHub Actions
- 9.2 Jenkins
- 9.3 Docker
- 9.4 Sharding and merging

## **10. Command Reference**

- Further reading

# 1. Introduction

Test automation only pays off when people can *understand* its results. A green tick is easy to read; a red cross without context sends somebody off to rerun the test locally, dig through CI logs and guess. **Allure Report** fixes that by turning raw results into an interactive web report with:

- a dashboard of pass/fail statistics and **trends across builds**;
- a **step-by-step** breakdown of each test with timings;
- **attachments**: screenshots, videos, traces, HTML snapshots, network logs, console output, JSON/CSV files;
- **business-level grouping** (epics → features → stories), severities, owners and links to issues;
- **retries, flaky-test** detection, custom **defect categories** and an execution **timeline**.

This tutorial walks through the `playwright-allure-showcase` project. It is both a working test suite (about 50 real tests of login, product browsing, search, cart, checkout and the storefront API) and a reference in which **every Allure feature has a test that demonstrates it**.

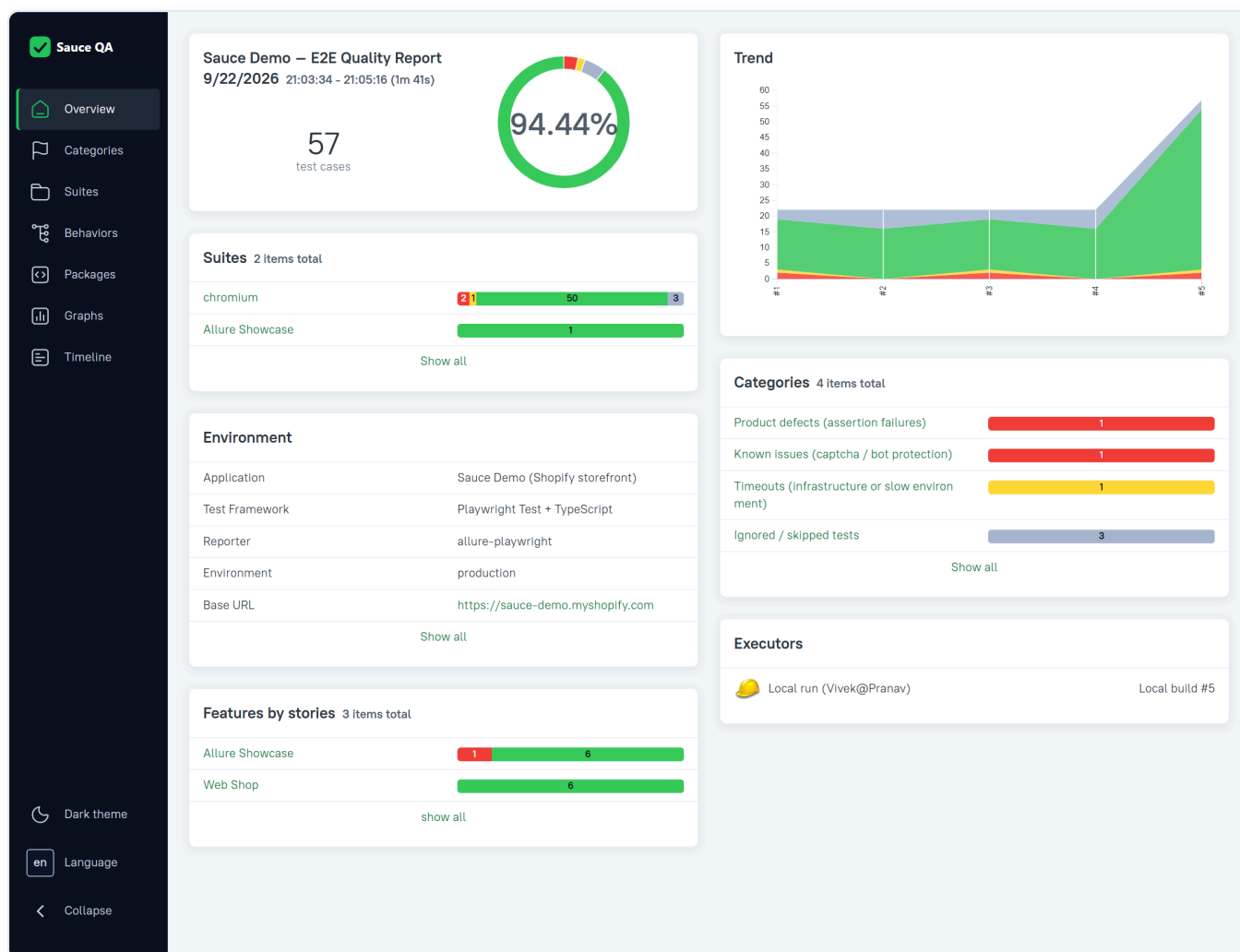


Figure 1 – The Allure Overview dashboard generated by this project

## 1.1 The system under test

The tests target <https://saucelabs.com>, a public Shopify demo store. It has 7 products (2 of them sold out), a customer login protected by **hCaptcha**, an AJAX cart API ( `/cart.js` , `/cart/add.js` ) and Shopify's hosted one-page checkout.

**Why hCaptcha matters.** Automated login submissions are intercepted by the captcha, so the store never shows a "wrong password" message to a bot. The login tests therefore assert what *can* be verified (the user stays signed out) and one showcase test documents the limitation as a **known issue**. Checkout tests stop before payment: no card data is ever entered, so no order can be placed.

## 1.2 How the pieces fit together

| Piece                                                            | Role                                                                                                                                |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Playwright Test</b>                                           | Runs the tests, provides fixtures, retries, screenshots, video and traces                                                           |
| <b>allure-playwright</b>                                         | A Playwright <i>reporter</i> that writes one JSON file per test result into <code>allure-results/</code>                            |
| <b>allure-js-commons</b>                                         | The runtime API used inside tests: <code>allure.step()</code> , <code>allure.epic()</code> , <code>allure.attachment()</code> , ... |
| <b>Allure Report 2</b> ( <code>allure-commandline</code> , Java) | Turns <code>allure-results/</code> into the HTML report in <code>allure-report/</code>                                              |



## 2. Setup & Installation

### 2.1 Prerequisites

| Tool           | Version                          | Why                                                  |
|----------------|----------------------------------|------------------------------------------------------|
| Node.js        | 20 or newer (22 LTS recommended) | Runs Playwright and the scripts                      |
| Java (JRE/JDK) | 8 or newer (17 recommended)      | The Allure 2 command-line tool is a Java application |
| Git            | any                              | Clone the project                                    |

Check them:

```
node -v          # v22.x
java -version    # openjdk version "17..."
```

BASH

### 2.2 Install the project

```
git clone <your-fork-url> playwright-allure-showcase
cd playwright-allure-showcase
npm ci                                # or: npm install
npx playwright install chromium      # add firefox webkit for cross-browser runs
cp .env.example .env                # optional: adjust settings
```

BASH

`npm ci` installs everything, including the Allure CLI (`allure-commandline`), so you do **not** need a global Allure installation.

### 2.3 Adding Allure to your own Playwright project

If you are starting from scratch, these are the only packages you need:

```
npm i -D @playwright/test allure-playwright allure-js-commons allure-commandline
```

BASH

Then register the reporter in `playwright.config.ts` (next chapter) and you are done.

## 2.4 Project structure

TEXT

```

playwright-allure-showcase/
├── src/
│   ├── tests/
│   │   ├── auth/          login.spec.ts, account-access.spec.ts
│   │   ├── products/      catalog, product-details, search, products-api
│   │   ├── cart/          cart.spec.ts
│   │   ├── checkout/      checkout.spec.ts
│   │   └── showcase/      allure-features.spec.ts, visual-evidence.spec.ts
│   ├── pages/             Page Object Model (BasePage, LoginPage, CartPage, ...)
│   ├── fixtures/
│   │   ├── base.fixture.ts page objects + logger + network/console monitors + evidence
│   │   └── data/           products.json, auth.json, checkout-customers.csv, ...
│   ├── utils/
│   │   ├── allure-helper.ts  typed facade over the Allure API + @step decorator
│   │   ├── logger.ts         per-test structured logger
│   │   ├── api-interceptor.ts network capture, timings, mocking
│   │   ├── console-monitor.ts browser console + uncaught errors
│   │   ├── shop-api-client.ts storefront API client with Allure steps
│   │   └── test-data.ts      JSON/CSV loaders
│   └── config/             environments, global setup/teardown, Allure metadata writer
├── scripts/               generate-report, trend-demo, run-and-report, docs tooling
├── allure-theme/          custom-theme.css + logo.svg
├── playwright.config.ts
├── allure-report.config.ts single source of truth for Allure settings
├── .github/workflows/     GitHub Actions pipeline
├── Jenkinsfile · docker/Dockerfile · docker-compose.yml
└── docs/                  this tutorial (Markdown + PDF) and screenshots

```

## 2.5 First run

BASH

```

npm run test:report    # run all tests, then ALWAYS generate the report
npm run allure:open    # open it in the browser

```

You should see about 57 results: ~50 real tests that pass, plus the showcase tests that **fail on purpose** (one failed, one broken, one known issue), one flaky test and three skipped tests. Set `SHOWCASE_FAILURES=false` to skip the intentional failures.

**Tip.** `npm run trend:demo` runs a fast subset four times and generates the report after each run, so you can see the **Trend** graphs straight away.

## 3. Basic Integration

### 3.1 Register the reporter

Allure is just another Playwright reporter. Keep the console reporter ( `list` ) so you still see progress in the terminal:

TS

```
// playwright.config.ts (excerpt)
import { defineConfig } from '@playwright/test';
import type { AllureReporter } from 'allure-playwright';
import allureConfig from './allure-report.config';
import { config } from './src/config';

const allureReporterOptions: AllureReporter['options'] = {
  resultsDir: allureConfig.resultsDir, // 'allure-results'
  // false = only test.step()/allure.step() in the report (clean);
  // true = also every Playwright API call and expect() (verbose).
  detail: config.allureDetail,
  suiteTitle: true,
  links: allureConfig.links, // issue/tms URL templates
  categories: allureConfig.categories, // custom defect categories
  globalLabels: { application: 'sauce-demo-storefront' },
};

export default defineConfig({
  testDir: './src/tests',
  retries: config.retries,
  reporter: [
    ['list'],
    ['allure-playwright', allureReporterOptions],
    ['html', { outputFolder: 'playwright-report', open: 'never' }],
  ],
  use: {
    baseURL: config.baseURL,
    screenshot: { mode: 'only-on-failure', fullPage: true },
    video: 'retain-on-failure',
    trace: 'retain-on-failure',
  },
  globalSetup: './src/config/global-setup.ts',
  globalTeardown: './src/config/global-teardown.ts',
});
```

Screenshots, videos and traces produced by Playwright are **attached to the Allure result automatically**; you do not have to write any code for them.

## 3.2 Reporter options

| Option                       | Meaning                                                                                                                                                                                                                       |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>resultsDir</code>      | Where result files are written (default <code>allure-results</code> )                                                                                                                                                         |
| <code>detail</code>          | <code>true</code> records every Playwright API call and <code>expect()</code> as a step. Great for debugging, noisy for stakeholders. This project defaults to <code>false</code> and exposes <code>ALLURE_DETAIL=true</code> |
| <code>suiteTitle</code>      | Use the file/describe title in the suite hierarchy                                                                                                                                                                            |
| <code>links</code>           | URL templates so <code>allure.issue('42')</code> becomes a clickable link                                                                                                                                                     |
| <code>categories</code>      | Custom defect categories (Categories tab)                                                                                                                                                                                     |
| <code>environmentInfo</code> | Static key/values for the Environment widget (this project writes them itself; see §5.12)                                                                                                                                     |
| <code>globalLabels</code>    | Labels added to every result                                                                                                                                                                                                  |

## 3.3 Generate, open and serve

BASH

```

npx allure generate allure-results -o allure-report --clean # build static HTML
npx allure open allure-report                             # serve it locally
npx allure serve allure-results                           # generate to a temp dir + open
npx allure generate --single-file allure-results -o out    # one self-contained HTML file

```

The project wraps these in npm scripts that also handle history, metadata and the theme:

| Command                                 | What it does                                              |
|-----------------------------------------|-----------------------------------------------------------|
| <code>npm test</code>                   | Run the suite (results in <code>allure-results/</code> )  |
| <code>npm run allure:generate</code>    | Metadata + history + <code>allure generate</code> + theme |
| <code>npm run allure:single-file</code> | Same, but a single shareable <code>index.html</code>      |
| <code>npm run allure:open</code>        | Open <code>allure-report/</code> in the browser           |
| <code>npm run allure:serve</code>       | Quick look without keeping a report                       |
| <code>npm run test:report</code>        | Run tests then always generate the report                 |
| <code>npm run report</code>             | Generate and open                                         |

**Why not just double-click `index.html` ?** The multi-file report loads its data with XHR, which browsers block on `file://` URLs. Use `allure open` (it starts a small web server) or generate a `--single-file` report.

## 3.4 Clean results between runs

allure-playwright never deletes old results; it keeps appending files. Without cleaning, a new report would mix results from several runs. The project's global setup wipes `allure-results/` at the start of each run

(history is kept separately in `allure-history/`):

```
// src/config/global-setup.ts
export default async function globalSetup(_: FullConfig): Promise<void> {
  const resultsDir = path.resolve(allureConfig.resultsDir);
  if (process.env.ALLURE_KEEP_RESULTS !== 'true' && fs.existsSync(resultsDir)) {
    fs.rmSync(resultsDir, { recursive: true, force: true });
  }
  fs.mkdirSync(resultsDir, { recursive: true });
}
```

Set `ALLURE_KEEP_RESULTS=true` when you deliberately merge results, for example from several CI shards.

## 4. Framework Architecture

### 4.1 Configuration management

All runtime settings live in `src/config/index.ts`, which reads `.env` and environment variables once. Environments are declared in `src/config/environments.ts`:

```
export const environments = {
  production: {
    name: 'production',
    baseUrl: 'https://saucе-demo.myshopify.com',
    actionTimeout: 15_000, navigationTimeout: 45_000, expectTimeout: 10_000,
    testTimeout: 90_000, slowRequestThresholdMs: 1_500, currencySymbol: '£',
  },
  staging: { /* same shape, longer timeouts */ },
} satisfies Record<string, EnvironmentConfig>;
```

TS

| Variable                                                          | Default                       | Effect                                                                    |
|-------------------------------------------------------------------|-------------------------------|---------------------------------------------------------------------------|
| <code>TEST_ENV</code>                                             | <code>production</code>       | Environment profile                                                       |
| <code>BASE_URL</code>                                             | profile value                 | Override the URL                                                          |
| <code>HEADLESS</code>                                             | <code>true</code>             | <code>false</code> shows the browser ( <code>npm run test:headed</code> ) |
| <code>SLOWMO</code>                                               | <code>0</code>                | Slow down every action (ms)                                               |
| <code>BROWSERS</code>                                             | <code>chromium</code>         | <code>all</code> adds firefox, webkit and a mobile profile                |
| <code>RETRIES</code> / <code>WORKERS</code>                       | 1 local, 2 CI / 3 local, 2 CI | Execution                                                                 |
| <code>VIDEO</code> , <code>TRACE</code> , <code>SCREENSHOT</code> | on failure                    | Evidence capture                                                          |
| <code>ALLURE_DETAIL</code>                                        | <code>false</code>            | Verbose Playwright steps in Allure                                        |
| <code>SHOWCASE_FAILURES</code>                                    | <code>true</code>             | Run the intentionally red tests                                           |
| <code>LOG_LEVEL</code>                                            | <code>warn</code>             | Console echo level (the full log is always attached)                      |

### 4.2 Page Object Model

Page objects encapsulate locators and actions. Each public action is an **Allure step** thanks to the `@step` decorator (§5.6):

TS

```
export class CartPage extends BasePage {
  protected readonly path = '/cart';
  readonly lines: Locator = this.page.locator('section#cart form
.row:has(input[name="updates[]"]')');
  readonly total: Locator = this.page.locator('section#cart .cart.total h2');

  @step('Set quantity of line {0} to {1} and update the cart')
  async setQuantity(lineIndex: number, quantity: number): Promise<void> {
    await this.lines.nth(lineIndex).locator('input[name="updates[]"]').fill(String(quantity));
    await this.updateButton.click();
    await this.page.waitForURL(/\/cart/);
  }
}
```

### 4.3 Fixtures: evidence for free

`src/fixtures/base.fixture.ts` extends Playwright's `test` with page objects, a logger, network and console monitors, an API client and an **automatic evidence fixture**. Every test gets its evidence attached without writing any code:

TS

```
evidence: [
  async ({ page, logger, network, consoleMonitor }, use, testInfo) => {
    await AllureHelper.contextParameters({ Viewport: '1366x900', Environment: config.name });
    logger.info(`▶ START "${testInfo.title}" (attempt ${testInfo.retry + 1})`);

    await use(); // ← the test runs here

    const failed = testInfo.status !== testInfo.expectedStatus;
    if (failed) await AllureHelper.attachHtmlSnapshot(page, 'HTML snapshot at failure');
    await network.stop();
    await network.attachToAllure({ full: failed }); // network log, API payloads, failures
    await consoleMonitor.attachToAllure(); // console messages + errors
    await AllureHelper.attachText('📄 execution.log', logger.toText());
  },
  { auto: true },
],
```

In the report, attachments added during fixture teardown appear inside the collapsible **After Hooks** section at the end of the test body.

Tests import `test` and `expect` from the fixture file instead of `@playwright/test`:

TS

```
import { expect, test } from '../../fixtures/base.fixture';

test('Adding a product updates the header cart counter', async ({ productPage, cartPage }) => {
  ... });
```

## 5. Allure Features Explained

Each section explains one capability, shows the code the project uses and the result in the report.

### 5.1 Titles and descriptions

The test title becomes the result name. Descriptions accept **Markdown** (`allure.description`) or **HTML** (`allure.descriptionHtml`):

```
test('Login page renders every element a customer needs to sign in', async ({ loginPage }) => {TS
  await AllureHelper.applyMetadata({
    story: 'Login form',
    severity: Severity.CRITICAL,
    description: [
      'Verifies the **customer login** page is reachable and complete.',
      '',
      '| Element | Expectation |',
      '|-----|',
      '| Email field | visible, `type=email` |',
    ].join('\n'),
  });
  // ...
});
```

`allure.displayName()` renames a result at runtime, for example to include data only known during execution.

### 5.2 Test statuses

Allure has five statuses. How allure-playwright maps Playwright outcomes is worth knowing:

| Allure status | Colour | Produced by (allure-playwright 3.x)                                      | Showcase test                                       |
|---------------|--------|--------------------------------------------------------------------------|-----------------------------------------------------|
| passed        | green  | test passed, or a <code>test.fail()</code> test failed as expected       | <code>EXPECTED FAILURE: known defect...</code>      |
| failed        | red    | test failed (assertion or any thrown error)                              | <code>FAILED: cart total assertion...</code>        |
| broken        | yellow | test <b>timed out</b>                                                    | <code>BROKEN: test exceeds its timeout...</code>    |
| skipped       | grey   | <code>test.skip()</code> , <code>test.fixme()</code> , conditional skips | <code>SKIPPED: ...</code> , <code>FIXME: ...</code> |
| unknown       | purple | results without a final status (rare)                                    |                                                     |

**Failed vs broken.** In Allure's vocabulary *failed* means "the product is wrong" (an assertion failed) and *broken* means "the test or its environment is wrong". allure-playwright 3.x reports every failing Playwright test as **failed** and only **timeouts** as **broken**. Steps are classified more finely: a step that throws a non-assertion error is shown as broken inside a failed test.

TS

```
test('BROKEN: test exceeds its timeout (infrastructure problem)', async ({ page }) => {
  test.setTimeout(8_000);
  await page.goto('/');
  await allure.step('Wait for a widget that never renders', async () => {
    await page.locator('#loyalty-widget-that-does-not-exist').waitFor({ timeout: 0 });
  });
});

test('SKIPPED: runs on WebKit only', async ({ browserName }) => {
  test.skip(browserName !== 'webkit', 'Apple Pay button only renders on WebKit (current: ${browserName})');
});

test('EXPECTED FAILURE: known defect marked with test.fail()', async ({ page }) => {
  test.fail(true, 'Issue #88: the store has no /pages/shipping page yet');
  const response = await page.goto('/pages/shipping');
  expect(response?.status()).toBe(200); // fails → reported as passed (expected)
});
```

showcase/allure-features.spec.ts › Allure feature showcase › Result statuses › FAILED: cart total assertion (Intentional product defect)

**Failed** **FAILED: cart total assertion (Intentional product defect)**

Overview History Retries

Error: Expected £49.50 after a 10% loyalty discount  
 expect(received).toBe(expected) // Object.is equality  
 Expected: 49.5  
 Received: 55

Tags: **showcase** **intentional-failure**

Categories: Product defects (assertion failures)

Severity: blocker

Duration: ⌚ 3s 234ms

Description

Fails on purpose: expects a 10% discount that the store does not apply. Shows the failure message, screenshot, HTML snapshot, video and trace—and both attempts in the Retries tab.

Owner

qa-enablement-team

Parameters

Environment: production

Project: chromium

Retry: 1

Viewport: 1366x900

Links

🐛 [Issue #404](#)

Execution

▼ Test body

|                                                                           |       |
|---------------------------------------------------------------------------|-------|
| ➤ API GET /products/grey-jacket.js 4 parameters, 1 sub-step, 1 attachment | 464ms |
| ➤ API POST /cart/add.js 4 parameters, 2 sub-steps, 2 attachments          | 775ms |
| ✅ Open CartPage (/cart)                                                   | 712ms |
| ✅ Read cart total                                                         | 16ms  |
| ➤ ✅ Verify: 10% loyalty discount is applied to the cart total             | 3ms   |

Figure 2 – A failed result shows the error message, the category it was sorted into, its metadata and the steps

Test timeout of 8000ms exceeded.

Tags: **showcase** **intentional-failure**

Categories: Timeouts (Infrastructure or slow environment)

Severity: normal

Duration: ⌚ 10s 606ms

Description

allure-playwright reports a timed-out test as broken (not failed): the problem is in the test/infra, not in the product.

Owner

qa-enablement-team

Parameters

Environment: production

Project: chromium

Retry: 1

Viewport: 1366x900

Execution

▼ Test body

|                                          |               |
|------------------------------------------|---------------|
| ➤ Wait for a widget that never renders   | 9s 194ms      |
| ➤ 📷 screenshot                           | 📄 400.3 KiB ↗ |
| ➤ 📺 video                                | 📄 648.3 KiB ↗ |
| ➤ 📄 error-context                        | 📄 14.2 KiB ↗  |
| ➤ 📄 trace                                | 📄 4.8 MiB ↗   |
| ➤ After Hooks 7 sub-steps, 7 attachments | 2s 606ms      |

Test timeout of 8000ms exceeded.

Figure 3 – A timed-out test is reported as broken

### 5.3 Behaviors: epic, feature, story

The **Behaviors** tab groups results by business capability instead of by file. Three labels drive it:

```
test.beforeEach(async () => {
  await AllureHelper.applyMetadata({ epic: 'Web Shop', feature: 'Checkout', owner: 'qa-checkout-team', layer: 'e2e' });
});

test('Pay now with an empty form lists every required field', async ({ cartPage, checkoutPage })
=> {
  await AllureHelper.applyMetadata({ story: 'Checkout validation', severity: Severity.CRITICAL
});
  // ...
});
```

Runtime calls made in `beforeEach` are applied to the test, which lets a whole `describe` share its epic, feature and owner.

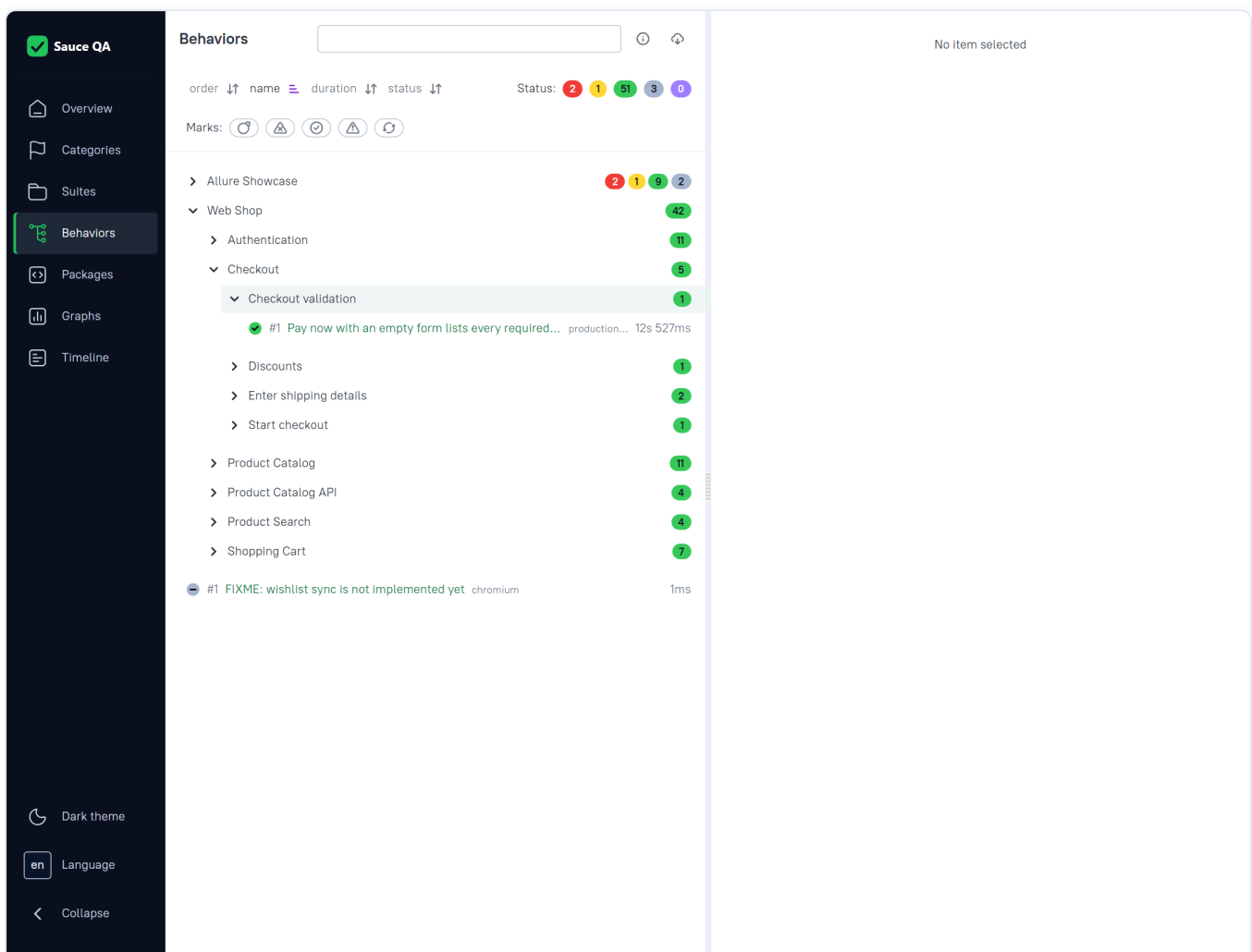


Figure 4 – Behaviors tab: epics, features and stories with status counts. This is the feature coverage view.

The **Features by stories** widget on the Overview page lists epics with the number of **features** under each (for example *Web Shop*: 6 = Authentication, Product Catalog, Product Search, Product Catalog API, Shopping

Cart, Checkout).

**Note.** Tests skipped with `test.fixme()` never run their hooks, so runtime labels are not applied and they appear at the root of the Behaviors tree. Use static annotations (§5.14) if skipped tests must be grouped too.

## 5.4 Severity

Five levels are available through the `Severity` enum and appear in the result header, filters and the **Severity** graph:

| Level                 | Use for                              | Example in the project                                   |
|-----------------------|--------------------------------------|----------------------------------------------------------|
| <code>blocker</code>  | Core business flow is impossible     | Catalog lists every product; add to cart; reach checkout |
| <code>critical</code> | Major function broken, no workaround | Checkout validation, product prices                      |
| <code>normal</code>   | Default                              | Search results, cart/API sync                            |
| <code>minor</code>    | Cosmetic or edge case                | Password recovery form, 404 page                         |
| <code>trivial</code>  | Barely noticeable                    | Header navigation                                        |

TS

```
await allure.severity(Severity.BLOCKER);
```

## 5.5 Tags, owner, layer, custom labels and suites

TS

```
test('Adding a product ... ', { tag: ['@smoke'] }, async () => { ... }); // Playwright tags →
Allure tags
await allure.tags('metadata', 'documentation'); // runtime tags
await allure.owner('qa-checkout-team');
await allure.layer('e2e'); // e2e | api | ui | ...
await allure.label('component', 'storefront'); // any custom label
await allure.parentSuite('Allure Showcase'); // override the Suites tree
await allure.suite('Metadata');
await allure.subSuite('Labels & links');
```

Playwright tags work for both tools: `npx playwright test --grep @smoke` selects tests and Allure shows the tags as chips you can filter on.

**Sauce QA**

**Suites**

order  $\uparrow\downarrow$  name  $\uparrow\downarrow$  duration  $\uparrow\downarrow$  status  $\uparrow\downarrow$  Status: 2 1 51 3 0

Marks:

- > Allure Showcase
  - chromium
    - > auth\account-access.spec.ts
      - 2
    - > auth\login.spec.ts
      - 9
    - > cart\cart.spec.ts
      - 7
      - > Shopping cart
        - 7
- > checkout\checkout.spec.ts
  - 5
- > products\catalog.spec.ts
  - 4
- > products\product-details.spec.ts
  - 7
- > products\products-api.spec.ts
  - 4
- > products\search.spec.ts
  - 4
- > showcase\allure-features.spec.ts
  - 2 1 7 3
- > showcase\visual-evidence.spec.ts
  - 1

**cart\cart.spec.ts > Shopping cart > Cart total equals the sum of all line totals**

Passed **Cart total equals the sum of all line totals**

Overview History Retries

Tags: regression cart

Severity: critical

Duration: 8s 006ms

Owner: qa-checkout-team

Parameters: production, chromium, 1366x900

Viewports: Grey Jacket, Striped top

Execution

- > Test body
  - Open product page "/products/grey-jacket" 901ms
  - Click "Add To Cart" 2s 132ms
  - Open product page "/products/striped-top" 591ms
  - Click "Add To Cart" 2s 179ms
  - Open CartPage (/cart) 722ms
  - Read cart lines 83ms
  - Read cart total 17ms
  - Verify: two lines are in the cart 2ms
  - Verify: total £105.00 = sum of lines £105.00 2ms
  - cart-lines.csv 87 B
  - Cart section 51.1 KiB
  - After Hooks 7 sub-steps, 7 attachments 775ms
  - stderr 501 B

Figure 5 – Suites tab: project → file → describe → test, with the selected result on the right

Tags: metadata documentation smoke showcase

Severity: critical

Duration: 2s 609ms

Description

Why this test exists  
It shows every kind of metadata Allure can hold. Open the tabs on the right:  
Labels: epic, feature, story, severity, owner, layer, component, tags  
Links: Issue, TMS, Jira (custom template), docs, related  
Suites: parentSuite / suite / subSuite overridden at runtime

Owner: qa-enablement-team

Parameters: production, chromium, 1366x900

Links: <https://github.com/your-org/playwright-allure-showcase/issues/7>, [Issue #101](#), [TC-5001](#), [Jira: SHOP-204](#), [Allure Playwright docs](#), [Related: API reference](#)

Execution

- > Test body
  - Open HomePage (/) 1s 359ms
  - Verify: home page title mentions Sauce Demo 56ms
  - After Hooks 5 sub-steps, 5 attachments 977ms

Figure 6 – Tags, severity, HTML description, owner, parameters and links (issue, TMS, Jira, docs, related)

## 5.6 Detailed test steps

Steps make a report readable by people who never saw the code. This project uses four techniques.

## 1. `allure.step()` with nesting and step parameters. Each step has its own duration:

```
await allure.step('Level 1: browse the catalog', async (s1) => {
  await s1.parameter('page', '/collections/all');
  await catalogPage.open();

  await allure.step('Level 2: inspect products', async (s2) => {
    const products = await catalogPage.getProducts();
    await s2.parameter('products found', String(products.length));

    await allure.step('Level 3: pick the cheapest in-stock item', async (s3) => {
      const cheapest = [...products].sort((a, b) => a.price - b.price)[0];
      await s3.displayName(`Level 3: cheapest item is "${cheapest.name}"
(${cheapest.priceText})`);
    });
  });
});
```

## 2. The `@step` decorator turns page-object methods into steps. `{0}` and `{1}` are replaced by the arguments, `{this.path}` by an instance property and `{class}` by the class name:

```
// src/utils/allure-helper.ts
export function step(title?: string) {
  return function <This, Args extends unknown[], Return>(
    target: (this: This, ...args: Args) => Promise<Return>,
    context: ClassMethodDecoratorContext<This, (this: This, ...args: Args) => Promise<Return>,
  ) {
    return async function (this: This, ...args: Args): Promise<Return> {
      const className = (this as { constructor: { name: string } }).constructor.name;
      const name = title
        ? title
        : .replace(/\{(\d+)\}/g, (_, i: string) => formatArg(args[Number(i)]))
        .replace(/\{this\.\w+\}/g, (_, prop: string) => formatArg((this as Record<string, unknown>)[prop]))
        .replace(/\{class\}/g, className)
        : `${className}.${String(context.name)}`;
      return allure.step(name, () => target.call(this, ...args));
    };
  };
}

// usage in a page object
@step('Open {class} ({this.path})')
async open(): Promise<void> { ... }
```

Playwright compiles TypeScript with support for standard (TC39) decorators, so no `experimentalDecorators` flag is needed.

**3. Assertion steps with explanatory messages.** `AllureHelper.verify()` wraps an assertion in a step named after *what* is validated, and `expect(value, message)` puts a business-level message into the error:

```
await AllureHelper.verify(`total £${total.toFixed(2)} = sum of lines £${expected.toFixed(2)}`,
() => {
  expect(total).toBeCloseTo(lines.reduce((s, l) => s + l.lineTotal, 0), 2);
});
await AllureHelper.verify('10% loyalty discount is applied to the cart total', () =>
  expect(total, 'Expected £49.50 after a 10% loyalty discount').toBe(49.5),
);
```

**4. Log steps** mark decisions or observations without a body, with any status:

```
await allure.logStep('Decision: guest checkout path chosen', Status.PASSED);
await allure.logStep('Optional recommendation widget not present – skipped', Status.SKIPPED);
await allure.logStep('Non-critical analytics beacon failed', Status.BROKEN, new Error('beacon
blocked'));
```

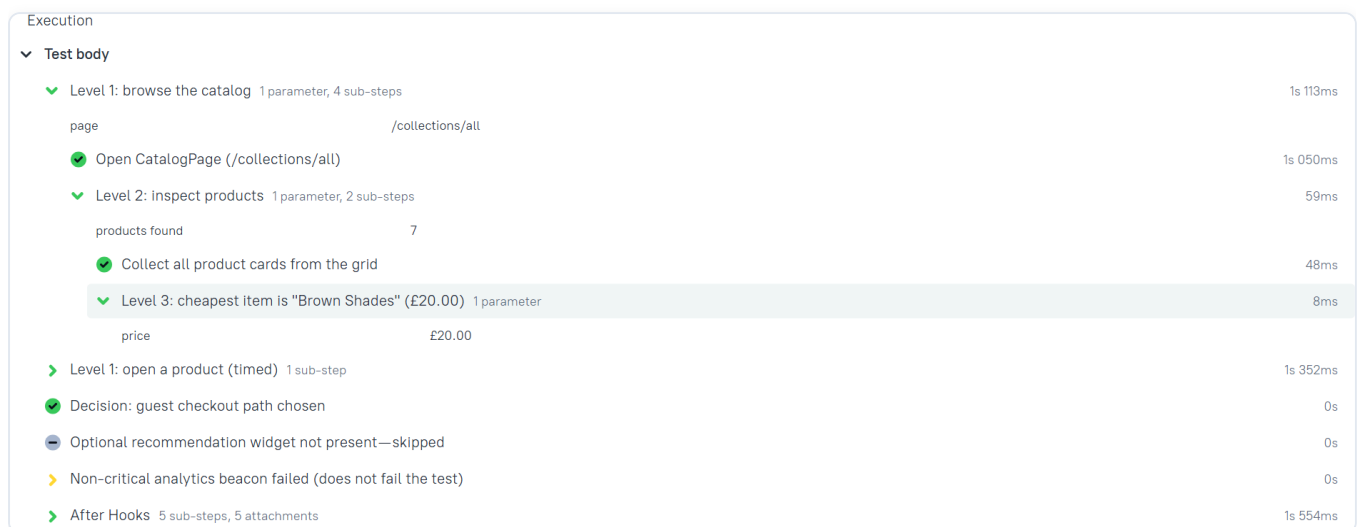


Figure 7 – Nested steps, step parameters, per-step timing and log steps with different statuses

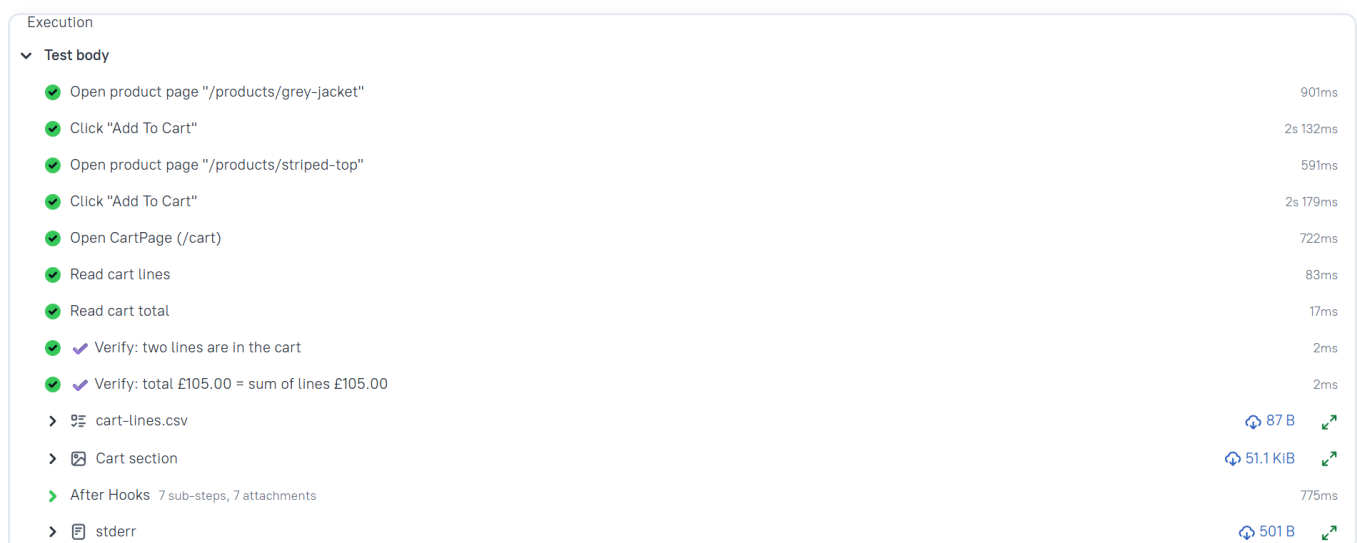


Figure 8 – Decorated page-object methods and ✓ Verify steps read like a test case

`detail: true` vs `false`. With `detail: true` allure-playwright also records every `page.click()`, `expect()` and fixture as a step. Keep it off for stakeholder reports and switch it on while debugging: `ALLURE_DETAIL=true npm test`.

## 5.7 Visual evidence: screenshots, video, traces, HTML snapshots

| Evidence               | How it is produced                                                              | Configured in                     |
|------------------------|---------------------------------------------------------------------------------|-----------------------------------|
| Screenshot on failure  | Playwright <code>screenshot: { mode: 'only-on-failure', fullPage: true }</code> | <code>playwright.config.ts</code> |
| Checkpoint screenshots | <code>AllureHelper.checkpoint(page, 'name')</code>                              | tests                             |
| Element screenshot     | <code>AllureHelper.attachElementScreenshot(locator, 'name')</code>              | tests                             |
| Video                  | Playwright <code>video: 'retain-on-failure'</code> (or <code>'on'</code> )      | config / <code>test.use()</code>  |
| Trace                  | Playwright <code>trace: 'retain-on-failure'</code>                              | config / <code>test.use()</code>  |
| HTML snapshot          | <code>page.content()</code> attached as <code>text/html</code> on failure       | evidence fixture                  |

```

async checkpoint(page: Page, name: string, options: { force?: boolean; fullPage?: boolean } =
  {}) {
  if (!config.checkpointScreenshots && !options.force) return;
  await allure.step('Checkpoint: ${name}', async () => {
    const png = await page.screenshot({ fullPage: options.fullPage ?? false, animations:
      'disabled' });
    await allure.attachment('Checkpoint - ${name}', png, ContentType.PNG);
  });
},

```

TS

To always record video for one scenario, set it at **file level**. Playwright refuses `test.use({ video })` inside a `describe` because it needs a new browser context:

```

// src/tests/showcase/visual-evidence.spec.ts
test.use({ video: { mode: 'on', size: { width: 1366, height: 900 } }, trace: 'on' });

```

TS

| Execution                                  |                            |               |
|--------------------------------------------|----------------------------|---------------|
| ▼ Test body                                |                            |               |
| ✓ Open HomePage (/)                        |                            | 684ms         |
| ➤ 📄 Checkpoint: 1 – Home page              | 1 sub-step, 1 attachment   | 114ms         |
| ✓ Go to the catalog                        |                            | 492ms         |
| ➤ 📄 Checkpoint: 2 – Catalog                | 1 sub-step, 1 attachment   | 88ms          |
| ✓ Open product "Grey jacket" from the grid |                            | 781ms         |
| ➤ 📄 Checkpoint: 3 – Product details        | 1 sub-step, 1 attachment   | 78ms          |
| ✓ Click "Add To Cart"                      |                            | 2s 126ms      |
| ✓ Open CartPage (/cart)                    |                            | 571ms         |
| ➤ 📄 Checkpoint: 4 – Cart                   | 1 sub-step, 1 attachment   | 114ms         |
| ✓ ✓ Verify: cart has one line              |                            | 16ms          |
| ➤ 📺 video                                  |                            | 📶 657.1 KiB 📶 |
| 📄 📺 trace                                  |                            | 📶 7.8 MiB 📶   |
| ➤ After Hooks                              | 7 sub-steps, 7 attachments | 1s 048ms      |
| ➤ 📄 stderr                                 |                            | 📶 188 B 📶     |

Figure 9 – Each checkpoint is a step with its screenshot attached

Screenshot

70.2 KiB

Search

About Us

Log In

Sign up

My Cart (1)

Check Out

simple.

Just a demo site showing off what Sauce can do.

Home

Catalog

Blog

About Us

Wish list

Refer a friend

f t i p s

Home — Cart

My Cart

| Description                                                                                                                                                                                                                                                                     | Price | Qty | Total |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-----|-------|--|
| <div><div></div><div><div>Grey jacket - Grey jacket</div><div>Sauce Demo</div><div>This area is populated by the product description. If you ever run out of things to say about what you're...</div></div></div> <div>£55.00</div> <div>1</div> <div>£55.00</div> <div>x</div> |       |     |       |  |

Continue Shopping

Total £55.00

Add a note to your order...

UPDATECHECK OUT

Footer

Search

About Us

About Us

This is a demo site created for [Sauce](#), an awesome new way to make your Shopify site social. Sauce allows you to let customers to share what they purchase to their friends, and see what their friends have purchased or "wanted" on your store.

MasterCard Visa Mastercard

Figure 10 – Playwright's failure screenshot, attached automatically

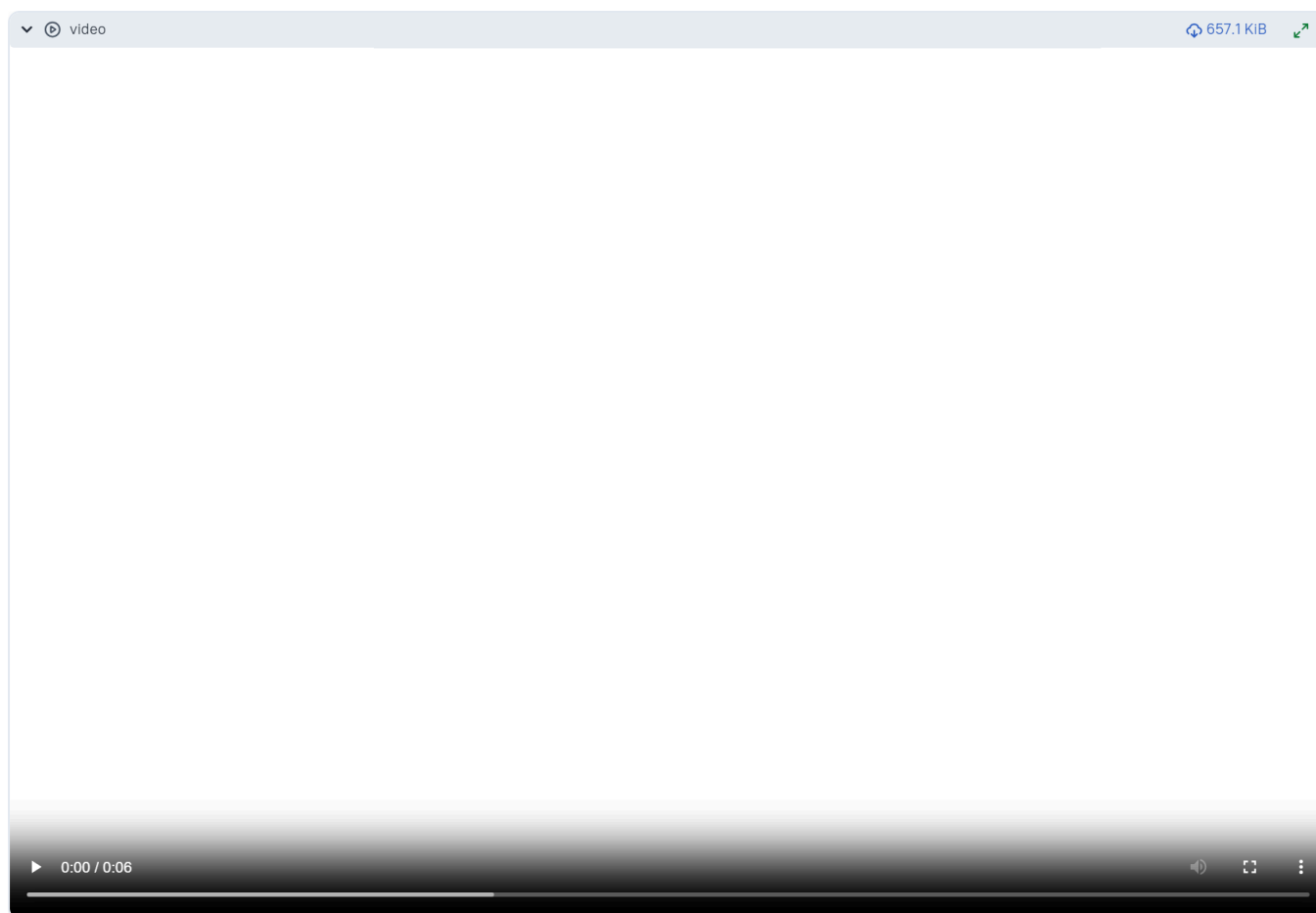


Figure 11 – The recorded video plays inside the report

The **trace** attachment ( `trace.zip` ) can be downloaded from the report and opened with `npx playwright show-trace trace.zip` or at <https://trace.playwright.dev>. It is the most powerful debugging artefact: DOM snapshots before and after every action, network, console and source.

## 5.8 Network and performance monitoring

`src/utils/api-interceptor.ts` listens to Playwright's page events and records every exchange:

```
start(): void {
  this.page.on('request', this.onRequest);           // method, url, type, first-party?
  this.page.on('response', this.onResponse);         // status, headers, JSON body (first-party
  XHR/fetch)
  this.page.on('requestfinished', this.onFinished); // duration from request.timing()
  this.page.on('requestfailed', this.onFailed);     // transport errors (net::ERR_*)
}
```

At the end of each test the evidence fixture attaches:

-  **Network summary** (JSON): totals, average and p95 time, slowest URL, counts by type and status;
-  **Network log** (HTML table): first-party requests and all failures, with 4xx/5xx/failed rows in red and slow rows in yellow;

- 🚫 **API calls with payloads:** request/response bodies of the store's AJAX API (sensitive headers are redacted);
- ❌ **Failed network requests**, only when there are any;
- `network-log.csv` for failed tests.

It can also **simulate failures** with `page.route()`, which is great for error-handling tests:

```
test('Add to cart fails gracefully when the cart service is unreachable', async ({ productPage, network, shopApi }) => {
  await network.simulateNetworkFailure('*/cart/add.js'); // route.abort()
  await productPage.openByHandle('grey-jacket');
  await productPage.clickAddToCart();

  await expect.poll(() => network.find('/cart/add.js').filter((e) => e.outcome === 'failed')).toHaveLength(0);
  const { body } = await shopApi.getCart();
  await AllureHelper.verify('server-side cart is still empty', () => expect(body.item_count).toBe(0));
});
```

Network log (first-party + failures) 16.7 KIB

HTTP traffic — 124 requests, avg 131ms, p95 362ms

| #  | method | status | time (ms) | type       | url                                                                                                             | note                    |
|----|--------|--------|-----------|------------|-----------------------------------------------------------------------------------------------------------------|-------------------------|
| 1  | GET    | 200    | 362       | document   | https://sauce-demo.myshopify.com/products/grey-jacket                                                           |                         |
| 2  | GET    | 200    | 43        | stylesheet | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/theme.scss?v=137440661423078363071656106300                |                         |
| 4  | GET    | 200    | 66        | script     | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/jquery.fancybox.js?v=35668124874407433581394656067         |                         |
| 5  | GET    | 200    | 67        | script     | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/jquery.elevatezoom.js?v=35333893780288212601394656066      |                         |
| 6  | GET    | 200    | 66        | script     | https://sauce-demo.myshopify.com/cdn/shopifycloud/storefront/assets/themes_support/option_selection-b017cd28.js |                         |
| 7  | GET    | 200    | 67        | script     | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/jquery.theme.js?v=42021284561027497791425319943            |                         |
| 8  | GET    | 200    | 64        | image      | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/logo.png?v=75283676530009549261394657629                   |                         |
| 9  | GET    | 200    | 65        | image      | https://sauce-demo.myshopify.com/cdn/shop/products/fleece.jpg?v=1394656989                                      |                         |
| 11 | GET    | 200    | 109       | image      | https://sauce-demo.myshopify.com/cdn/shop/products/fleece_grande.jpg?v=1394656989                               |                         |
| 12 | GET    | 200    | 113       | image      | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/cc-amex.png?v=85206210438157468371394656065                |                         |
| 13 | GET    | 200    | 153       | image      | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/cc-visa.png?v=60401960785579442311394656066                |                         |
| 14 | GET    | 200    | 160       | script     | https://sauce-demo.myshopify.com/checkouts/internal/preloads.js?locale=en-AU                                    |                         |
| 15 | POST   | 200    | 226       | ping       | https://sauce-demo.myshopify.com/well-known/shopify/monorail/unstable/produce_batch                             |                         |
| 16 | POST   | 200    | 268       | ping       | https://sauce-demo.myshopify.com/well-known/shopify/monorail/unstable/produce_batch                             |                         |
| 18 | GET    | 200    | 238       | script     | https://sauce-demo.myshopify.com/cdn/shopifycloud/storefront/assets/storefront/load_feature-1bd60354.js         |                         |
| 19 | GET    | 200    | 225       | script     | https://sauce-demo.myshopify.com/cdn/shopifycloud/perf-kit/shopify-perf-kit-3.9.4.min.js                        |                         |
| 20 | GET    | 200    | 223       | image      | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/cc-mastercard.png?v=163639441997642668931394656066         |                         |
| 21 | GET    | failed | 710       | script     | https://dk7vxmpleem9z.cloudfront.net/sgmnt.min.js                                                               | net::ERR_BLOCKED_BY_ORB |
| 22 | GET    | 200    | 89        | script     | https://sauce-demo.myshopify.com/cdn/wpm/baa86ff40wb21a529ap5b81b74dm8e0f4ce5m.js                               |                         |
| 23 | GET    | 200    | 84        | script     | https://sauce-demo.myshopify.com/cdn/s/trekkie.storefront.222108d476a0ee628f5c8f99419136812de1982a.min.js       |                         |
| 24 | GET    | 200    | 90        | script     | https://sauce-demo.myshopify.com/cdn/shopifycloud/storefront/assets/shop_events_listener-4e26a9ce.js            |                         |
| 27 | GET    | 200    | 126       | image      | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/sprite.png?v=19773218180732711381394656067                 |                         |
| 28 | GET    | 200    | 122       | image      | https://sauce-demo.myshopify.com/cdn/shop/t/2/assets/sprite.png?v=197732181807327113813946560675                |                         |
| 29 | POST   | 200    | 333       | fetch      | https://sauce-demo.myshopify.com/api/collect                                                                    | net::ERR_ABORTED        |
| 30 | GET    | failed | 261       | script     | https://www.marinsoftware.com/                                                                                  | net::ERR_BLOCKED_BY_ORB |
| 31 | POST   | 200    | 312       | ping       | https://sauce-demo.myshopify.com/well-known/shopify/monorail/unstable/produce_batch                             |                         |

Figure 12 – The network log: status codes, response times, the aborted /cart/add.js request

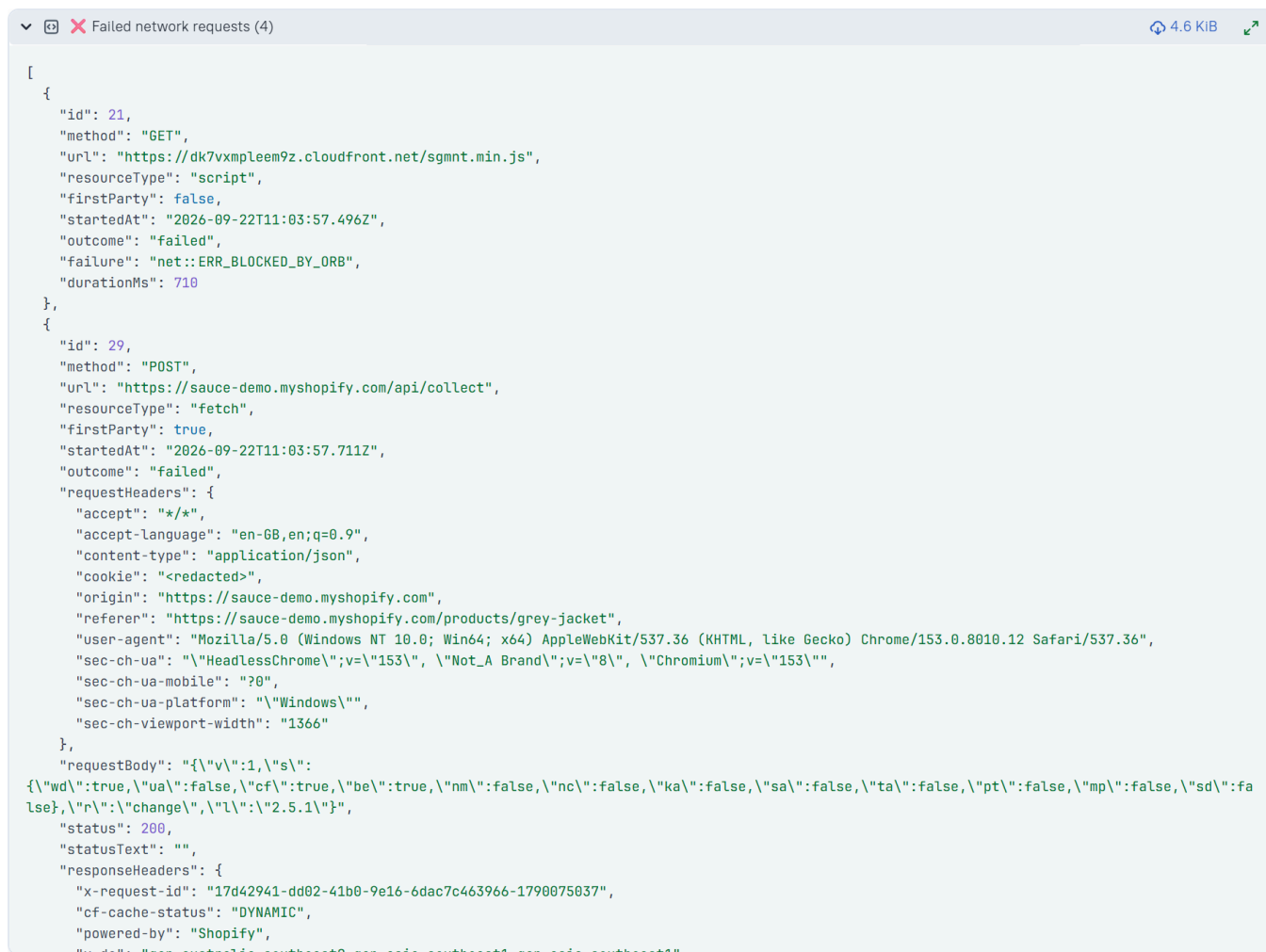


Figure 13 – Failed requests with the transport error, marked as mocked

**Page performance.** The showcase measures Navigation Timing (TTFB, DOMContentLoaded, load, transfer size) for key pages and attaches it as CSV, with each measurement a step with parameters:

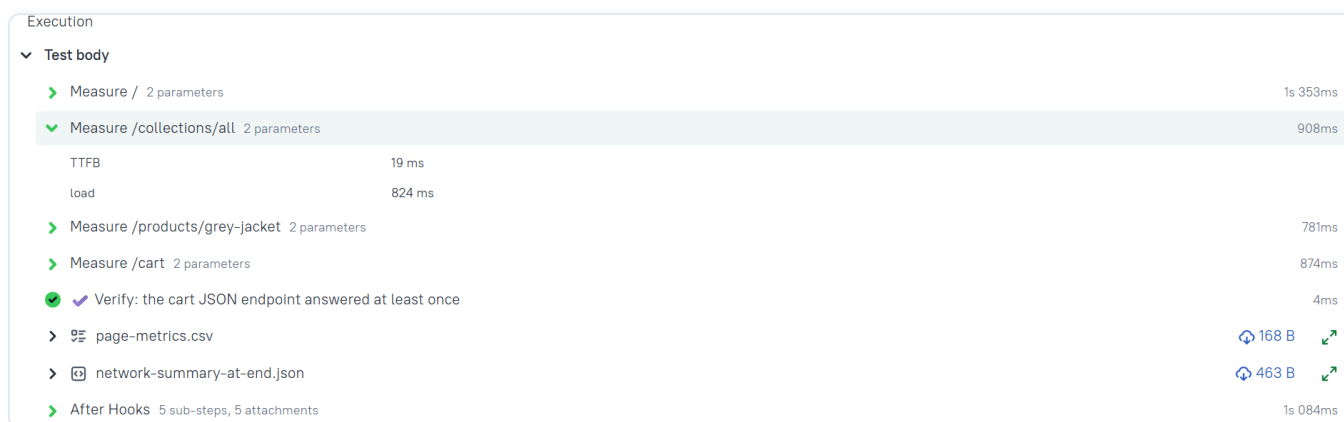


Figure 14 – One step per page with TTFB and load time as step parameters

## 5.9 API testing with Allure steps

`ShopApiClient` wraps Playwright's `APIRequestContext`. Every call is a step with the method, URL, status and duration as parameters and the request/response bodies attached:

```
private async call<T>(method: 'GET' | 'POST', url: string, data?: unknown) {
  return allure.step(`API ${method} ${url}`, async (ctx) => {
    await ctx.parameter('method', method);
    await ctx.parameter('url', url);
    if (data !== undefined) await AllureHelper.attachJson(`Request ${method} ${url}`, data);

    const started = performance.now();
    const response = await this.raw(method, url, data);
    const durationMs = Math.round(performance.now() - started);
    await ctx.parameter('status', String(response.status()));
    await ctx.parameter('duration', `${durationMs} ms`);
    // ... parse, attach the response, throw on non-2xx
  });
}
```

Created from `page.request`, the client **shares cookies with the browser**, so the UI tests use it for fast setup: the checkout tests fill the cart through the API in `beforeEach` and only exercise the checkout in the UI.

The screenshot displays an Allure report entry for an API GET step. The step is titled 'API GET /products.json' and includes 4 parameters, 1 sub-step, and 1 attachment. The parameters shown are: method (GET), url (/products.json), status (200), and duration (406 ms). Below the parameters, the response is expanded, showing a JSON object with a list of products. The first product is 'Brown Shades' with a price of 20.00. The response is attached as 'Response 200 /products.json (406 ms)' with a size of 17.2 KiB.

```
{
  "products": [
    {
      "id": 407143389,
      "title": "Brown Shades",
      "handle": "brown-shades",
      "body_html": "",
      "published_at": "2015-02-02T13:36:00+00:00",
      "created_at": "2015-02-02T13:37:51+00:00",
      "updated_at": "2026-09-22T11:44:24+01:00",
      "vendor": "Sauce Demo",
      "product_type": "Sunglasses",
      "tags": [],
      "variants": [
        {
          "id": 1063105029,
          "title": "Default Title",
          "option1": "Default Title",
          "option2": null,
          "option3": null,
          "sku": "",
          "requires_shipping": true,
          "taxable": true,
          "featured_image": null,
          "available": false,
          "price": "20.00",
          "grams": 0,
          "compare_at_price": null,
          "position": 1,
          "product_id": 407143389,
          "created_at": "2015-02-02T13:37:51+00:00",
          "updated_at": "2026-09-22T11:44:24+01:00"
        }
      ]
    },
    ...
  ],
  "images": [
    ...
  ]
}
```

Figure 15 – An API step: parameters (method, url, status, duration) and the JSON response

## 5.10 Console capture and logging

`ConsoleMonitor` records `console.*` messages and uncaught exceptions (`pageerror`). The evidence fixture attaches the full console output and a highlighted table of errors and warnings. Against the live store it regularly catches real problems, such as third-party 403s and a `TypeError` thrown by a theme script.

```
test('Browser console capture and analysis', async ({ page, consoleMonitor }) => {
  await page.goto('/');
  await page.evaluate(() => console.error('[demo] recommendation widget failed to load'));
  await AllureHelper.verify('the demo error was captured', () =>
    expect(consoleMonitor.errors().map((e) => e.text)).toContain('[demo] recommendation widget failed to load'),
  );
  await AllureHelper.verify('no uncaught JavaScript exceptions on first-party pages', () =>
    expect(consoleMonitor.getPageErrors().filter((e) =>
      e.pageUrl.startsWith(config.baseUrl))).toHaveLength(0),
  );
});
```

The per-test **Logger** buffers structured entries (levels, context and data) and attaches them as `execution.log`. `logger.data('cart.total', 105)` records variable values for debugging. Only warnings and errors are echoed to the terminal by default (`LOG_LEVEL`), and allure-playwright also attaches the test's stdout/stderr.

Console errors & warnings (2) 903 B

| time         | type    | message                                      | source | page                              |
|--------------|---------|----------------------------------------------|--------|-----------------------------------|
| 11:04:43.214 | warning | [demo] deprecated API used by a theme script |        | https://sauce-demo.myshopify.com/ |
| 11:04:43.214 | error   | [demo] recommendation widget failed to load  |        | https://sauce-demo.myshopify.com/ |

Figure 16 – Console errors and warnings with their source location

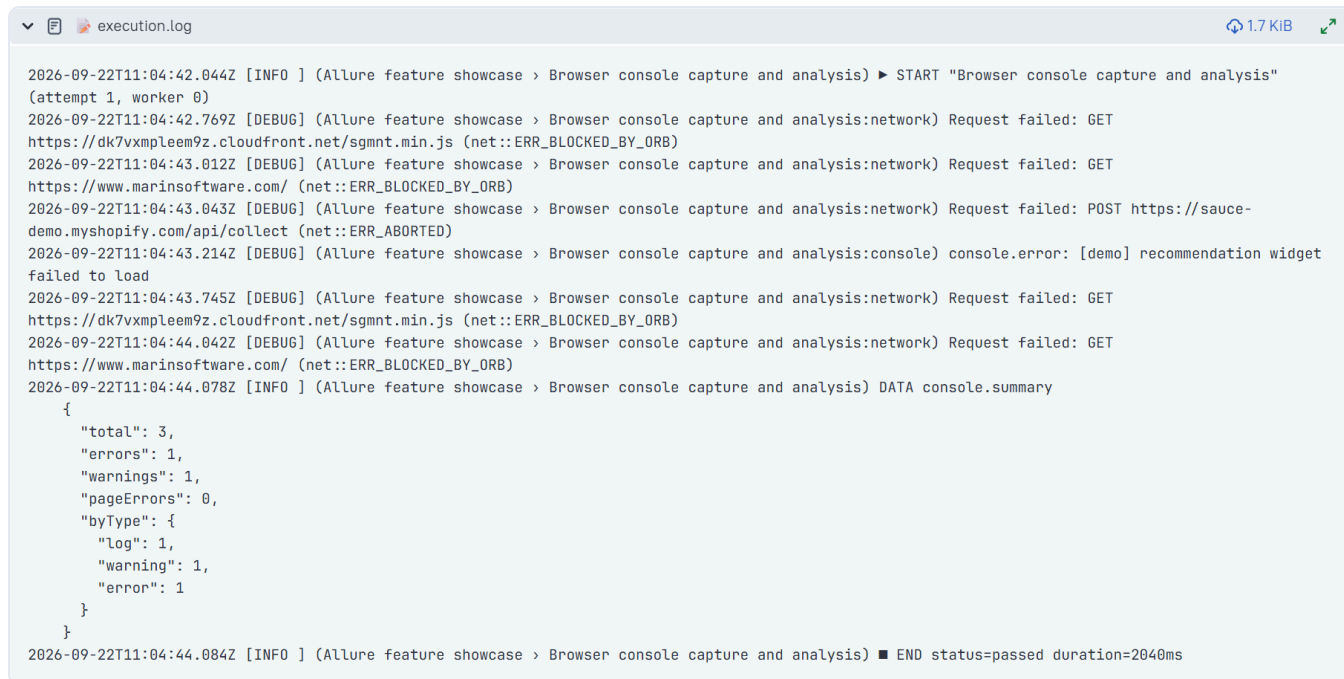


Figure 17 – The structured execution log, including logged test data

## 5.11 Parameters and test data

Parameters describe *what data* a result ran with. Allure uses them to tell data-driven variants apart:

```

await allure.parameter('Customer tier', 'Gold'); // plain
await allure.parameter('API key', 'sk_live_51H8secret', { mode: 'masked' }); // shown as
*****
await allure.parameter('Internal trace id', 'trace-7f3a9', { mode: 'hidden' }); // not shown
at all
await allure.parameter('Run timestamp', new Date().toISOString(), { excluded: true }); // not
part of history id

```

**excluded** does not hide a parameter. It only removes it from the *history id*, so values that change on every run (timestamps, retry numbers, viewports) do not split a test's history. Use **mode: 'hidden'** to hide it.

**Data-driven tests** generate one Allure result per data row. The source file is attached as evidence:

TS

```
const customers = loadCsv<Customer>('checkout-customers.csv');

for (const customer of customers) {
  test('Shipping details are accepted for ${customer.label}', async ({ cartPage, checkoutPage })
⇒ {
    await allure.parameter('customer', customer.label);
    await allure.parameter('email', customer.email, { mode: 'masked' });
    await AllureHelper.attachCsv('checkout-customers.csv (data source)', readDataFile('checkout-
customers.csv'));
    // ...
  });
}
```

## Owner

qa-enablement-team

## Parameters

API key: \*\*\*\*\*

Customer tier: Gold

Environment: production

Project: chromium

Run timestamp: 2026-09-22T11:04:36.689Z

Viewport: 1366x900

Figure 18 – Plain, masked and excluded parameters (the hidden one is not displayed)

## 5.12 Environment and executor information

The **Environment** widget reads `allure-results/environment.properties`, and the **Executors** widget reads `executor.json`. The project writes both in global teardown ( `src/config/allure-metadata.ts` ) so they can include **runtime** data such as the exact browser version, which each worker records:

TS

```
browserMetadata: [
  async ({ browser, browserName }, use, workerInfo) ⇒ {
    fs.mkdirSync(META_DIR, { recursive: true });
    fs.writeFileSync(path.join(META_DIR, `browser-${workerInfo.project.name}.json`),
      JSON.stringify({ project: workerInfo.project.name, browserName, browserVersion:
browser.version() }));
    await use();
  },
  { auto: true, scope: 'worker' },
],
```

The resulting `environment.properties` looks like this:

```

Environment=production
Base\ URL=https\://sauce-demo.myshopify.com
OS=Windows_NT 10.0.26200 (x64)
Node.js=v22.18.0
Headless=true
Viewport=1366x900
Retries=1
Browser\ (chromium)=chromium 153.0.8010.12

```

`executor.json` detects GitHub Actions and Jenkins (build name, number and URL) and falls back to a local executor with an incrementing build number. The build number labels the x-axis of the trend charts.

Per test, the evidence fixture adds *excluded* parameters (viewport, environment), and allure-playwright adds the Playwright **Project** (browser) as a parameter, so cross-browser results of the same test are distinguishable.

## 5.13 Retries and flaky tests

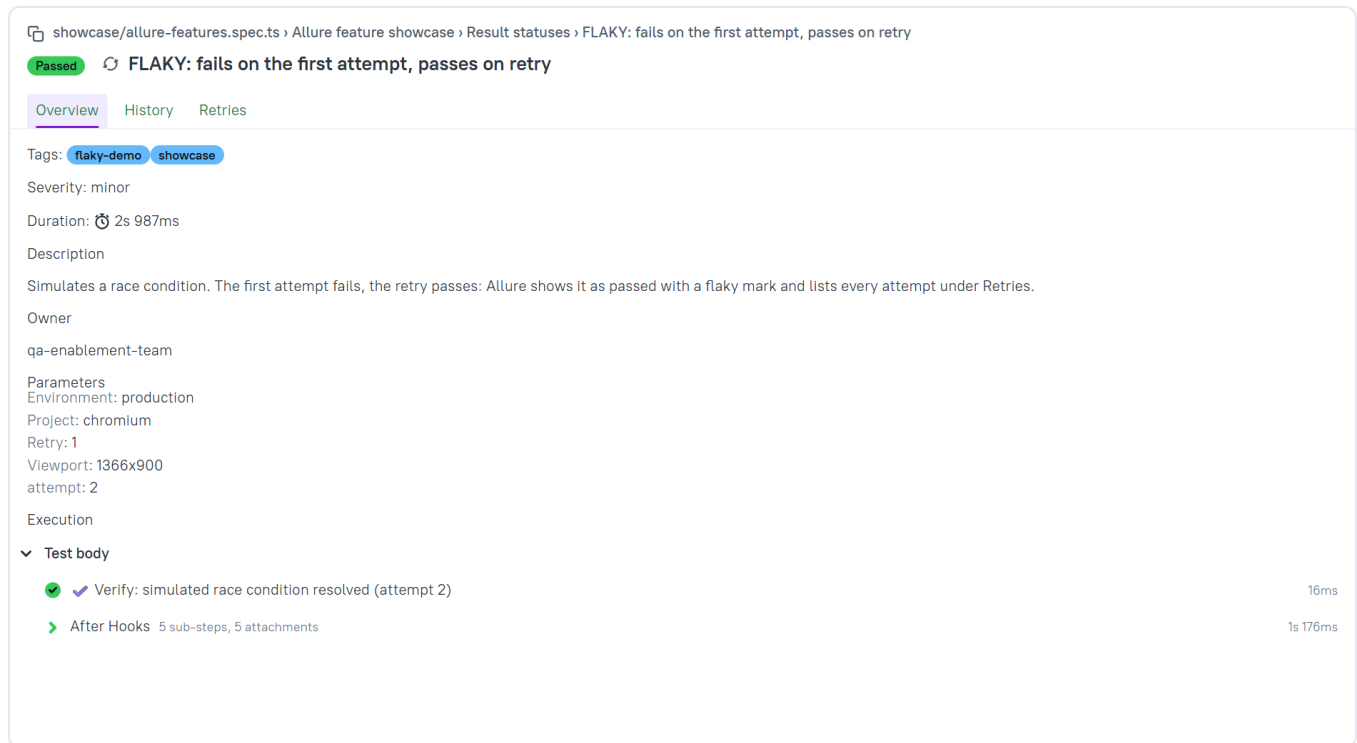
Playwright retries are configured with `retries` (1 locally, 2 on CI in this project). Allure groups all attempts of a test into **one** result; earlier attempts appear in the **Retries** tab. A test that failed and then passed is marked **flaky**.

TS

```

test('FLAKY: fails on the first attempt, passes on retry', async ({ page }, testInfo) => {
  await allure.parameter('attempt', String(testInfo.retry + 1), { excluded: true });
  await page.goto('/');
  await AllureHelper.verify(`simulated race condition resolved (attempt ${testInfo.retry + 1})`,
() =>
  expect(testInfo.retry, 'first attempt loses the simulated race').toBeGreaterThan(0),
  );
});

```



showcase/allure-features.spec.ts › Allure feature showcase › Result statuses › FLAKY: fails on the first attempt, passes on retry

**Passed** 🔄 **FLAKY: fails on the first attempt, passes on retry**

Overview History Retries

Tags: **flaky-demo** **showcase**

Severity: minor

Duration: ⌚ 2s 987ms

Description

Simulates a race condition. The first attempt fails, the retry passes: Allure shows it as passed with a flaky mark and lists every attempt under Retries.

Owner

qa-enablement-team

Parameters

Environment: production

Project: chromium

Retry: 1

Viewport: 1366x900

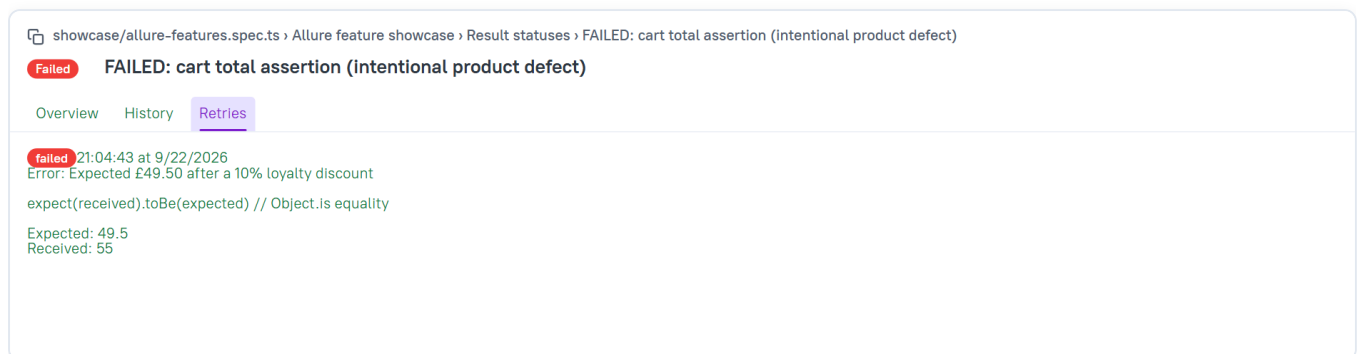
attempt: 2

Execution

▼ Test body

- ✓ Verify: simulated race condition resolved (attempt 2) 16ms
- After Hooks 5 sub-steps, 5 attachments 1s 176ms

Figure 19 – A flaky result: passed, with the flaky mark and the retry count



showcase/allure-features.spec.ts › Allure feature showcase › Result statuses › FAILED: cart total assertion (intentional product defect)

**Failed** **FAILED: cart total assertion (intentional product defect)**

Overview History **Retries**

**failed** 21:04:43 at 9/22/2026

Error: Expected £49.50 after a 10% loyalty discount

expect(received).toBe(expected) // Object.is equality

Expected: 49.5

Received: 55

Figure 20 – The Retries tab lists every earlier attempt with its own status, duration and evidence

allure-playwright adds a **Retry** parameter (excluded from history) to retried attempts. Use the **Marks** filter (flaky, new failed, new broken, retries) in the tree views to find them quickly.

## 5.14 Links and relationships

Links connect a result to the rest of your tooling. Templates in `allure-report.config.ts` turn short ids into URLs:

```
links: {
  issue: { urlTemplate: 'https://github.com/your-org/playwright-allure-showcase/issues/%s',
    nameTemplate: 'Issue #s' },
  tms: { urlTemplate: 'https://your-org.testrail.io/index.php?cases/view/%s', nameTemplate:
    'TC-%s' },
  jira: { urlTemplate: 'https://your-org.atlassian.net/browse/%s', nameTemplate: '%s' },
},
```

TS

TS

```

await allure.issue('101'); // → Issue #101
await allure.tms('5001'); // → TC-5001
await allure.link('SHOP-204', 'Jira: SHOP-204', 'jira'); // custom type with its own
template
await allure.link('https://allurereport.org/docs/playwright/', 'Allure Playwright docs',
'docs');
await allure.link('https://allurereport.org/docs/playwright-reference/', 'Related: API
reference', 'related');

```

Metadata can also be declared **statically** with Playwright annotations, which is useful for tests that are skipped before any code runs:

TS

```

test('Rich metadata: labels, links, severity and descriptions', {
  tag: ['@smoke'],
  annotation: [
    { type: 'allure.label.feature', description: 'Report metadata' },
    { type: 'allure.label.story', description: 'Labels and links' },
    { type: 'issue', description: 'https://github.com/your-org/playwright-allure-
showcase/issues/7' },
  ],
}, async ({ homePage }) => { ... });

```

**Relationships between tests** are expressed through shared labels and links. Tests with the same **story** form a group in Behaviors; the same **issue** or **tms** id links tests that cover the same requirement; **allureId** / **testCaseId** give results a stable identity for TMS integrations; and **related** links point to other tests or specifications.

## 5.15 History and trend analysis

Allure keeps a **history/** folder inside each generated report. When the **next** report is generated, the previous **history/** must be copied into **allure-results/** first. That single step enables:

- the **Trend**, **Duration trend**, **Retries trend** and **Categories trend** graphs;
- the **History** tab of every test (its last runs with status and duration);
- the **new failed** / **new broken** / **flaky** marks.

**scripts/generate-report.ts** automates it:

TS

```

const restored = copyDir(path.join(historyDir, 'history'), path.join(resultsDir, 'history'));
// before
await runAllure(['generate', resultsDir, '-o', reportDir, '--clean', '--name',
allureConfig.reportName]);
copyDir(path.join(reportDir, 'history'), path.join(historyDir, 'history'));
// after

```

Allure always adds the *current* results as a new point on top of the restored history. To re-render the same results (for example as a single file) without counting them twice, run `npx tsx scripts/generate-report.ts --single-file --no-history-save`: the script then restores the history that was the *input* of the last build (`allure-history/previous-history/`) and does not save a new one.

On CI the history lives on the `gh-pages` branch (GitHub Actions) or is managed by the Allure Jenkins plugin (§9).

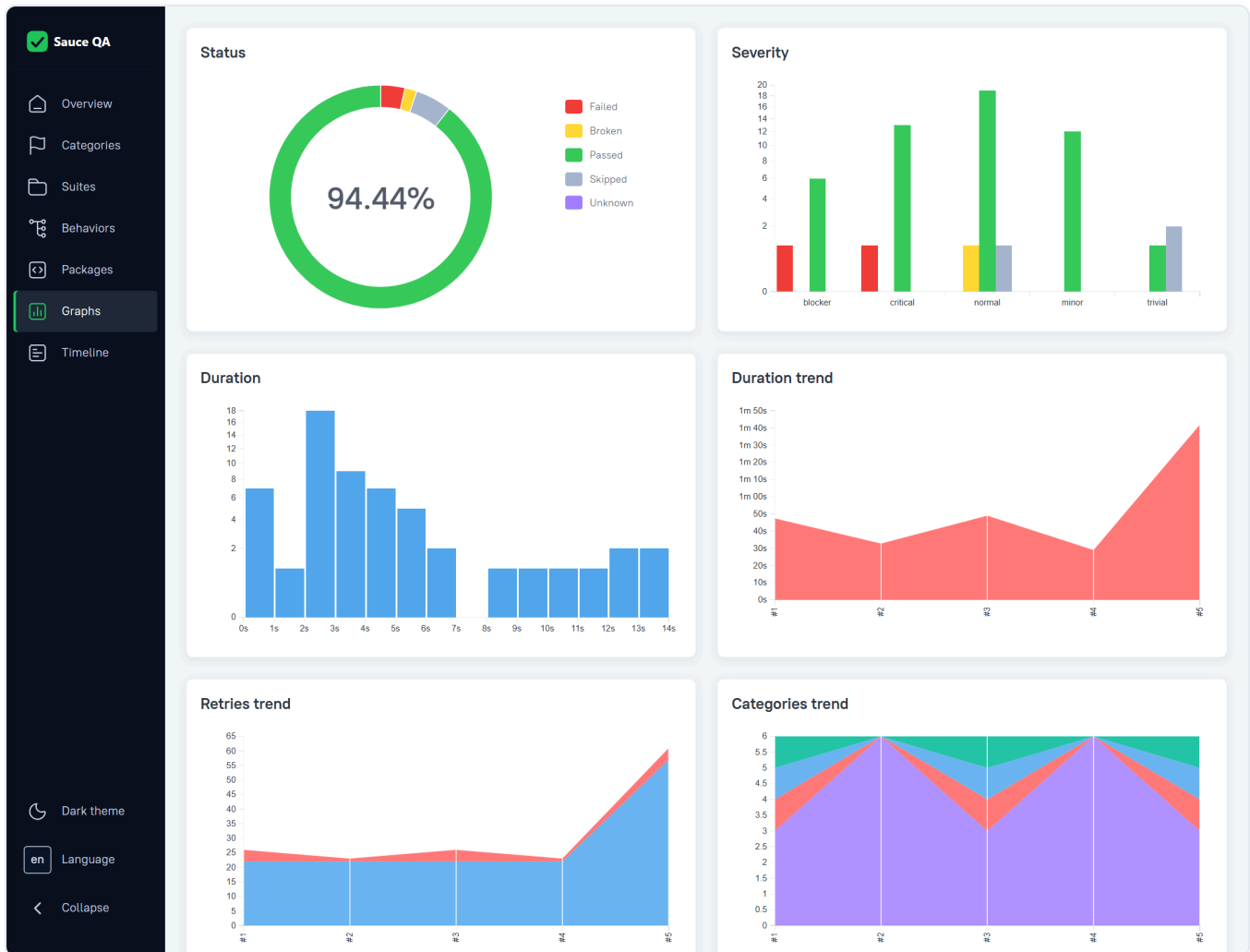


Figure 21 – Graphs: status, severity, duration distribution and the trend graphs built from history

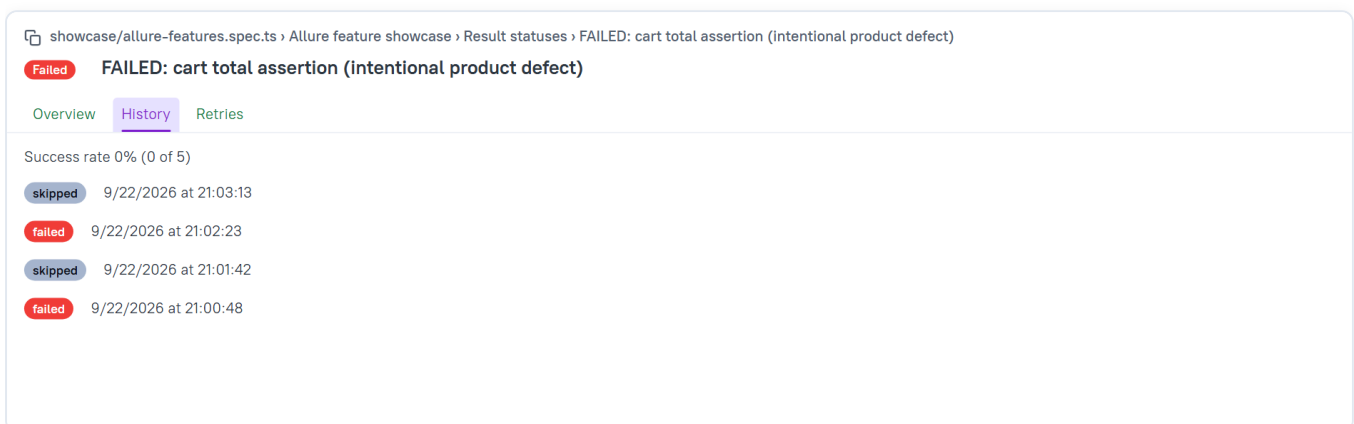


Figure 22 – The History tab shows how a single test behaved in previous builds

## 5.16 Custom attachments

Any content can be attached; the content type decides how Allure renders it:

| Content type                                                 | Rendering                           | Helper                                                          |
|--------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------|
| <code>application/json</code>                                | pretty-printed, collapsible         | <code>AllureHelper.attachJson()</code>                          |
| <code>text/csv</code>                                        | table                               | <code>AllureHelper.attachCsv()</code>                           |
| <code>text/plain</code>                                      | monospace text                      | <code>AllureHelper.attachText()</code>                          |
| <code>text/html</code>                                       | rendered HTML<br>(sandboxed iframe) | <code>AllureHelper.attachHtml()</code>                          |
| <code>image/png</code> , <code>image/svg+xml</code>          | image                               | <code>allure.attachment(name, buf, ContentType.PNG)</code>      |
| <code>video/webm</code>                                      | video player                        | automatic (Playwright video)                                    |
| <code>application/xml</code> ,<br><code>text/uri-list</code> | text / clickable links              | <code>allure.attachment(..., ContentType.XML / URI)</code>      |
| any file on disk                                             | download or preview                 | <code>allure.attachmentPath(name, path, { contentType })</code> |

TS

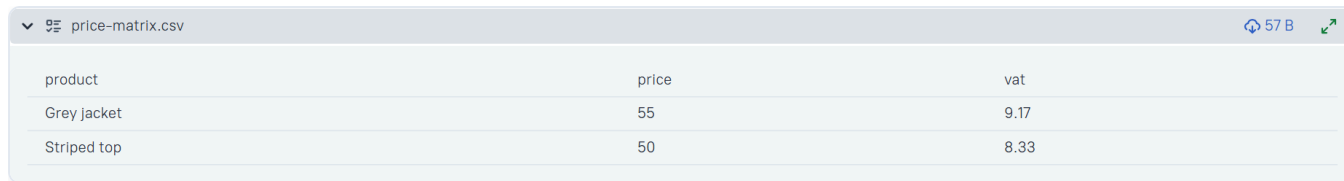
```
await AllureHelper.attachJson('order-payload.json', { orderId: 'S0-10042', items: [...],
currency: 'GBP' });
await AllureHelper.attachCsv('price-matrix.csv', [{ product: 'Grey jacket', price: 55, vat: 9.17
}, ...]);
await allure.attachment('sitemap-extract.xml', '<urlset>...</urlset>', ContentType.XML);
const logFile = testInfo.outputPath('application.log');
fs.writeFileSync(logFile, '[INFO] cart service ready\n[WARN] slow response 1800ms');
await AllureHelper.attachFile('application.log (file)', logFile, ContentType.TEXT);
await testInfo.attach('playwright-native-attachment.txt', { body: '...', contentType:
'text/plain' }); // also lands in Allure
```

### Execution

#### Test body

|                                                               |              |
|---------------------------------------------------------------|--------------|
| ✓ Open CartPage (/cart)                                       | 978ms        |
| ➤ 🖼️ Checkpoint: Viewport screenshot 1 sub-step, 1 attachment | 184ms        |
| ➤ 📄 order-payload.json                                        | 📄 140 B ↗    |
| ➤ 📄 price-matrix.csv                                          | 📄 57 B ↗     |
| ➤ 📄 sitemap-extract.xml                                       | 📄 60 B ↗     |
| ➤ 📄 rendered-receipt.html                                     | 📄 76 B ↗     |
| ➤ 📄 status-badge.svg                                          | 📄 240 B ↗    |
| ➤ 🔗 related-urls.uri                                          | 📄 86 B ↗     |
| ➤ 📄 application.log (file)                                    | 📄 74 B ↗     |
| ➤ 📄 playwright-native-attachment.txt                          | 📄 31 B ↗     |
| ➤ 🖼️ Element screenshot — logo                                | 📄 2 KiB ↗    |
| ➤ 📄 HTML snapshot (https://saucedemo.myshopify.com/cart)      | 📄 52.1 KiB ↗ |
| ➤ After Hooks 5 sub-steps, 5 attachments                      | 939ms        |

Figure 23 – JSON, CSV, XML, HTML, SVG, URI list, a log file, screenshots and an HTML snapshot



| product     | price | vat  |
|-------------|-------|------|
| Grey Jacket | 55    | 9.17 |
| Striped top | 50    | 8.33 |

Figure 24 – CSV attachments are rendered as tables

## 5.17 Categories: automatic defect triage

Categories sort failures by **message or stack-trace regex** and status. The first matching rule wins:

```
categories: [
  { name: 'Known issues (captcha / bot protection)', messageRegex: '.*(captcha|hCaptcha|bot protection).*',
    matchedStatuses: [Status.FAILED, Status.BROKEN] },
  { name: 'Timeouts (infrastructure or slow environment)', messageRegex: '.*(Timeout|timed out|exceeded).*',
    matchedStatuses: [Status.BROKEN, Status.FAILED] },
  { name: 'Network / API failures', messageRegex: '.*(net::ERR|ECONNREFUSED|status code|HTTP [45]\\d\\d|API).*',
    matchedStatuses: [Status.FAILED, Status.BROKEN] },
  { name: 'Product defects (assertion failures)', messageRegex: '.*(expect|Expected|toBe|toHave|toEqual|toContain).*',
    matchedStatuses: [Status.FAILED] },
  { name: 'Test defects (broken tests)', matchedStatuses: [Status.BROKEN] },
  { name: 'Flaky tests', matchedStatuses: [Status.PASSED, Status.FAILED, Status.BROKEN], flaky: true },
  { name: 'Ignored / skipped tests', matchedStatuses: [Status.SKIPPED] },
],
```

The *KNOWN ISSUE* showcase test fails with the message "hCaptcha bot protection prevents automated sign-in", so it lands in its own category instead of mixing with real product defects.

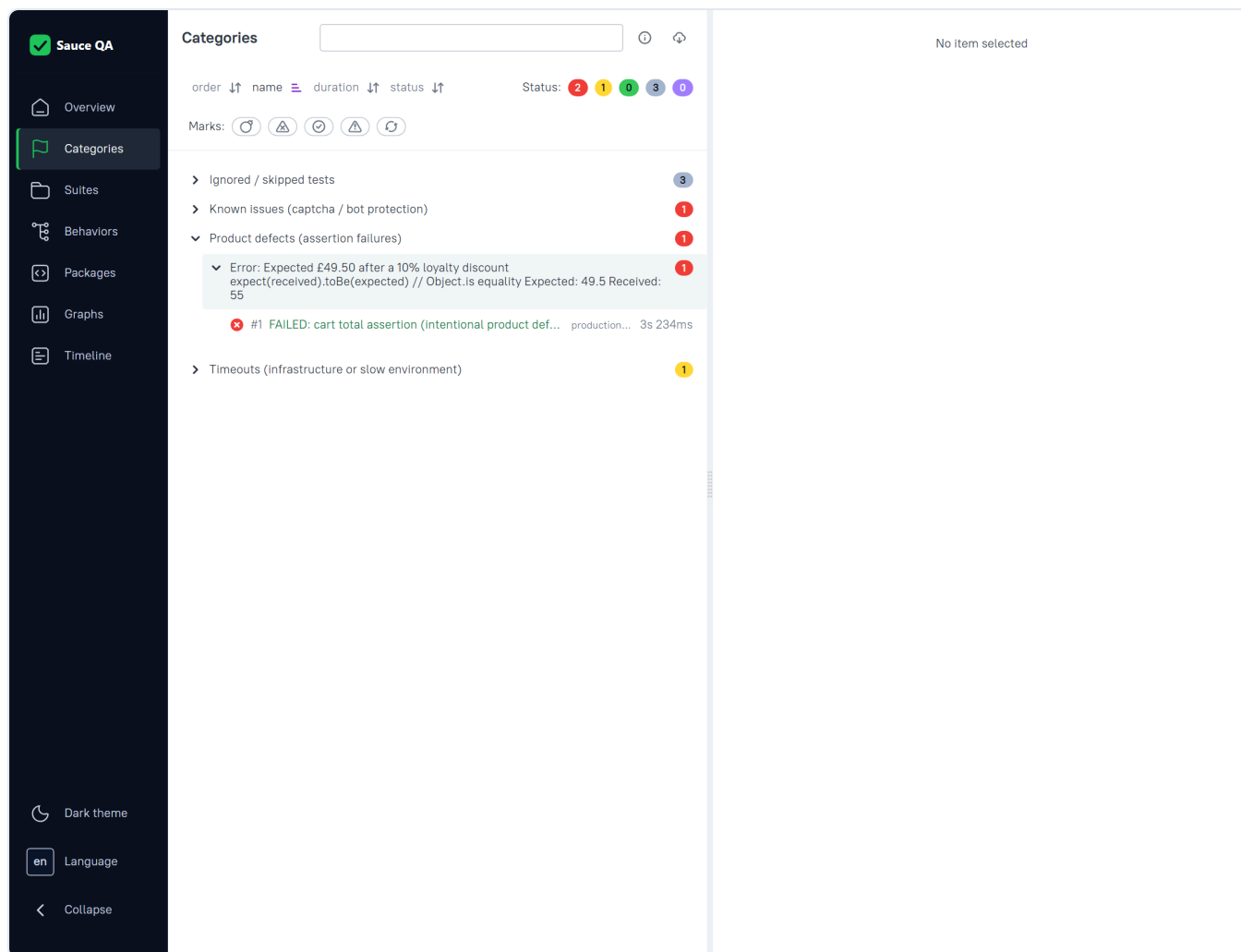


Figure 25 – Categories: failures grouped by rule, then by error message

## 5.18 Timeline and Packages

The **Timeline** shows when each test ran on which worker, which is useful for spotting slow tests, idle workers and parallelisation problems. **Packages** groups results by file path.

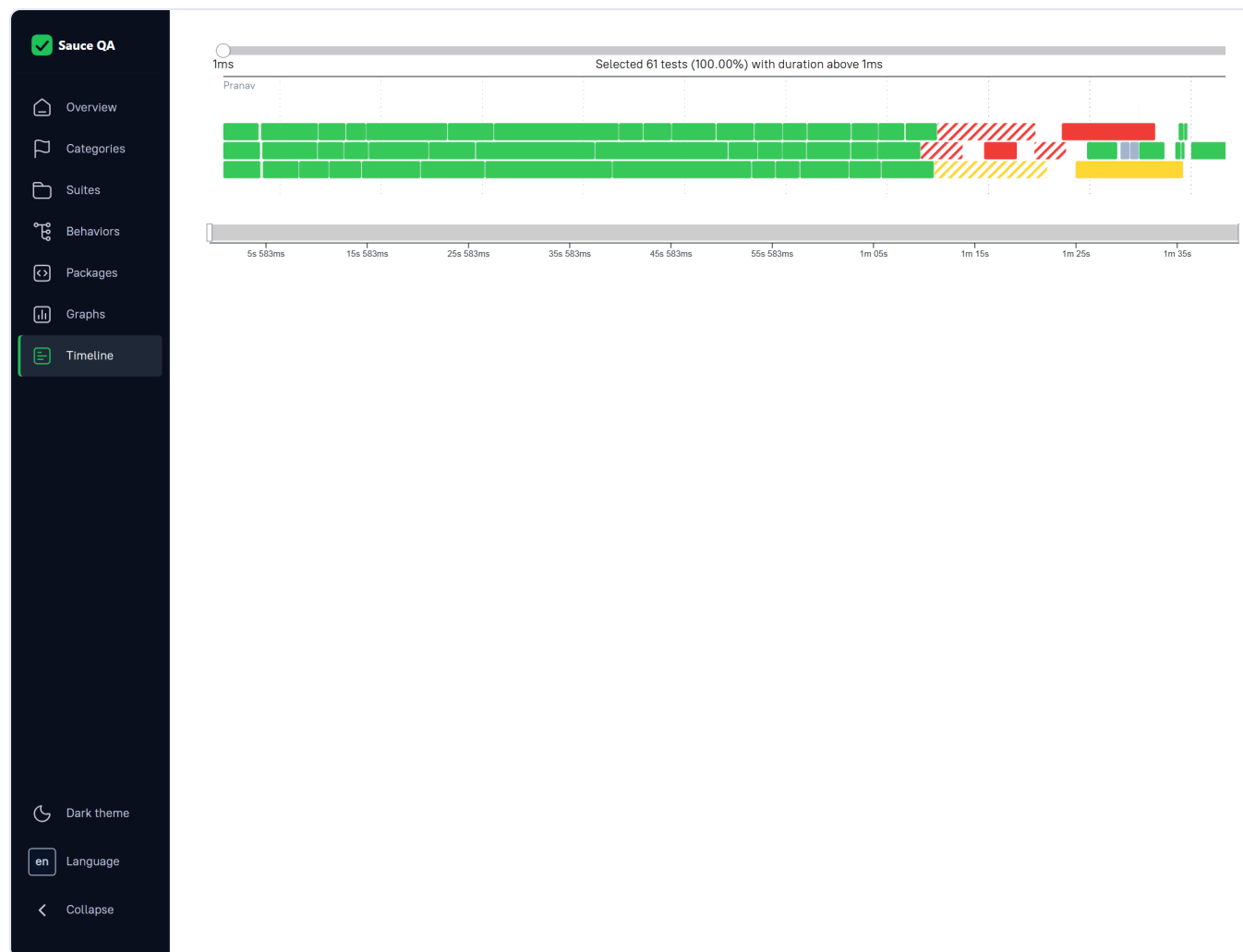


Figure 26 – Timeline: one lane per host/worker, one bar per test

**Sauce QA**

**Packages**

order ↓↑ name duration ↓↑ status ↓↑ Status: 2 1 51 3 0

Marks: [Icons]

> playwright-allure-showcase 2 1 51 3 0

No item selected

Figure 27 – Packages: results grouped by directory and spec file

## 6. Best Practices

---

### 6.1 Write reports for humans

- **Name tests after behaviour**, not implementation: "Cart total equals the sum of all line totals", not "test\_cart\_03".
- **Use steps as the test case.** A reader should understand the scenario from the step list alone. Decorate page-object methods and name verification steps after the business rule.
- **Put the "why" in assertion messages:** `expect(total, 'Expected £49.50 after a 10% loyalty discount')`.
- **Keep `detail: false` by default;** turn it on while debugging.

### 6.2 Metadata discipline

- Agree on a small, fixed set of **epics and features** that mirror the product. Put them in `beforeEach` so a `describe` shares them.
- Give every test a **severity** and an **owner** so failures are routed quickly.
- Link tests to **issues/TMS ids** with templates instead of hard-coded URLs.
- Use **tags** for run selection ( `@smoke`, `@regression` ) and keep them in the title's `tag` option.

### 6.3 Evidence without noise

- Attach evidence **automatically in fixtures**, not by hand in each test.
- Prefer `retain-on-failure` for video and trace: full evidence for failures, small reports for green runs.
- Capture **checkpoint screenshots** at meaningful moments only, and disable them in large suites ( `CHECKPOINT_SCREENSHOTS=false` ).
- **Redact secrets:** mask parameters ( `mode: 'masked'` ) and strip `cookie` / `authorization` headers from network logs (done in `api-interceptor.ts` ).
- Truncate large bodies; the report is a static site that people download.

### 6.4 Stable history

- Keep **test titles stable:** the history id is derived from the test's full name and its non-excluded parameters.
- Mark volatile parameters as `excluded: true` .
- Always restore `history/` before generating on CI, or the trends reset.
- Clean `allure-results/` before each run (except when merging shards).

### 6.5 Flakiness

- Use retries to **surface** flakiness, not to hide it. Review the *Flaky* marks and the *Retries trend* regularly.
- Wait for the real signal (the `/cart/add.js` response, a URL change, a visible element) instead of fixed timeouts. See `ProductPage.addToCart()` .

## 6.6 Speed

- Set up state through the **API** ( `shopApi.addToCart` ) and test only the behaviour under test through the UI.
- Run fully parallel with isolated browser contexts; the Timeline shows whether workers are well used.

## 7. Troubleshooting

| Symptom                                                                          | Cause                                                                                    | Solution                                                                                                                                                                                            |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ERROR: JAVA_HOME is set to an invalid directory</code>                     | <code>JAVA_HOME</code> points to a missing folder or to <code>.../bin</code>             | Point it at the JDK root. The project's scripts repair it for their own process ( <code>scripts/lib/allure-cli.ts</code> ): they fall back to the parent folder or to <code>java</code> on the PATH |
| <code>allure: command not found</code> / <code>'allure' is not recognized</code> | CLI not installed globally                                                               | Use <code>npx allure ...</code> or the npm scripts; <code>allure-commandline</code> is a dev dependency                                                                                             |
| Report shows <b>no tests</b> or old tests                                        | Results never written, or results from several runs mixed                                | Check <code>allure-results/</code> contains <code>*-result.json</code> ; clean it before each run (global setup does)                                                                               |
| <b>No results after passing <code>--reporter=...</code> on the CLI</b>           | A CLI <code>--reporter</code> replaces all reporters from the config                     | Do not pass <code>--reporter</code> , or pass both: <code>--reporter=list,allure-playwright</code>                                                                                                  |
| Blank page when opening <code>index.html</code>                                  | <code>file://</code> blocks the report's XHR requests                                    | <code>npm run allure:open</code> , <code>allure serve</code> , or <code>--single-file</code>                                                                                                        |
| <b>Trend graphs empty</b>                                                        | <code>history/</code> was not copied into <code>allure-results/</code> before generating | Use <code>npm run allure:generate</code> ; on CI restore the history (see §9)                                                                                                                       |
| Every test counts as "new" in history                                            | Titles or non-excluded parameters change between runs                                    | Keep titles stable; mark volatile parameters <code>excluded: true</code>                                                                                                                            |
| Expected "broken", got "failed"                                                  | allure-playwright 3 reports all failures as failed; only timeouts are broken             | See §5.2; use Categories to separate infrastructure errors by message                                                                                                                               |
| <code>Cannot use({ video }) in a describe group</code>                           | Browser/context options need a new worker                                                | Put <code>test.use({ video, trace })</code> at the top of the file                                                                                                                                  |
| Labels set in <code>beforeEach</code> missing on skipped tests                   | <code>test.fixme()</code> / declarative skip never runs hooks                            | Use static annotations ( <code>allure.label.*</code> )                                                                                                                                              |
| Excluded parameter still visible                                                 | <code>excluded</code> only affects the history id                                        | Use <code>mode: 'hidden'</code>                                                                                                                                                                     |
| Login tests can't sign in                                                        | Store is protected by hCaptcha                                                           | Expected: assert "not authenticated" (§1.1). For real apps use a test-mode captcha key or an API login                                                                                              |
| Attachment too big / slow report                                                 | Full-page screenshots or long bodies on every step                                       | Use viewport screenshots, truncate bodies, <code>retain-on-failure</code> video                                                                                                                     |
| <code>Executable doesn't exist ... ms-playwright</code>                          | Browsers not installed for this Playwright version                                       | <code>npx playwright install chromium</code> (add <code>--with-deps</code> on Linux CI)                                                                                                             |

| Symptom                                    | Cause                      | Solution                                                                                                                                                            |
|--------------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test data variables look wrong in failures | Hard to see runtime values | Use <code>logger.data('name', value)</code> and read  <code>execution.log</code> |

## 8. Advanced Customization

### 8.1 One configuration file

`allure-report.config.ts` holds the results/report/history folders, report name and language, link templates, categories, static environment entries and theme settings. `playwright.config.ts`, the metadata writer and the report scripts all import it, so there is a single place to change.

### 8.2 Custom theme and logo

Allure 2 has no theming API, but the report is a static web app. `generate-report.ts` injects `allure-theme/custom-theme.css` into `index.html` as an inline `<style>` (so it also works for `--single-file` reports), with `url(logo.svg)` replaced by an inline data URI of `allure-theme/logo.svg`:

```

:root { --brand: #22c55e; --nav-bg: #0b1220; --nav-hover: #1e293b; --nav-text: #cbd5e1; }

.side-nav { background: var(--nav-bg) !important; }
.side-nav__brand { background: url(logo.svg) no-repeat 14px center !important; background-size:
130px 35px !important; }
.side-nav__brand-icon, .side-nav__brand-text { visibility: hidden; }
.side-nav__icon { color: var(--nav-text) !important; } /* icons are
stroke="currentColor" */
.side-nav__link_active { box-shadow: inset 3px 0 0 var(--brand); color: #fff !important; }
.app__content a:not(.button) { color: #15803d; }

```

Edit the CSS and re-apply it without regenerating: `npm run tsx scripts/docs/inject-theme.ts`. Recent Allure 2 releases (2.46 is used here) also have a built-in **dark theme** (sidebar toggle):



Figure 28 – The built-in dark theme combined with the custom logo

Allure ships a `custom-logo-plugin` that you can enable in `allure.yml` and pass with `allure generate --config`, but CSS injection is simpler and survives `--single-file`.

## 8.3 Report name, language and filters

```
allure generate allure-results -o allure-report --clean --name "Sauce Demo - E2E Quality Report" --lang en
```

In the tree views you can filter by **status**, by **marks** (flaky, new failed, new broken, retries) and by test name. Sort by order, name, duration or status. The download icon exports the tree as CSV.

## 8.4 Global labels and listeners

```
globalLabels: { application: 'sauce-demo-storefront' }, // added to every result
listeners: [{ beforeTestResultWrite: (result) => { /* mutate or enrich results */ } }],
```

## 8.5 Running a subset with a test plan

allure-playwright supports Allure TestOps test plans: set `ALLURE_TESTPLAN_PATH=testplan.json` to run only the listed tests (by `allureId` or full name).

## 8.6 Allure Report 3

Allure Report 3 (`npm install allure` from the `allure` npm package) is a TypeScript rewrite that does not need Java. It reads the same `allure-results/`, so this framework works with it unchanged. Configure it with an `allurerc.mjs`:

```
// allurerc.mjs (Allure 3)
import { defineConfig } from 'allure';

export default defineConfig({
  name: 'Sauce Demo - E2E Quality Report',
  output: './allure-report',
  historyPath: './allure-history/history.jsonl', // built-in history, no copying needed
  plugins: { awesome: { options: { singleFile: false, reportLanguage: 'en' } } },
});
```

Allure 2 remains the default here because of its mature Categories, Graphs and Timeline pages and its Jenkins plugin.

## 9. CI/CD Integration

### 9.1 GitHub Actions

`.github/workflows/playwright-allure.yml` runs on push, pull requests, a weekday schedule and on demand:

1. install Node 22, Java 17, dependencies and browsers;
2. run the tests ( `continue-on-error` , so the report is always produced);
3. check out `gh-pages` and copy its `history/` into `allure-history/` ;
4. `npm run allure:generate` (the executor picks up the run number and URLs);
5. upload the report and raw results as artifacts;
6. publish `allure-report/` to GitHub Pages (main branch only);
7. fail the job if the tests failed.

YAML

```
- name: Restore Allure history
  uses: actions/checkout@v4
  continue-on-error: true
  with: { ref: gh-pages, path: gh-pages }

- name: Generate Allure report (with trends)
  run: |
    mkdir -p allure-history
    if [ -d gh-pages/history ]; then cp -r gh-pages/history allure-history/history; fi
    npm run allure:generate

- name: Publish to GitHub Pages
  if: github.ref == 'refs/heads/main'
  uses: peaceiris/actions-gh-pages@v4
  with: { github_token: '${{ secrets.GITHUB_TOKEN }}', publish_branch: gh-pages, publish_dir:
allure-report }
```

Enable GitHub Pages for the `gh-pages` branch once in the repository settings. The CI run sets `SHOWCASE_FAILURES=false` so the pipeline stays green.

### 9.2 Jenkins

Install the **Allure Jenkins plugin** and configure an *Allure Commandline* tool. The plugin generates the report, keeps history between builds, writes the executor and adds an "Allure Report" link and a trend chart to the job page:

GROOVY

```

pipeline {
  agent { dockerfile { filename 'docker/Dockerfile'; args '--ipc=host' } }
  stages {
    stage('Install') { steps { sh 'npm ci' } }
    stage('Test')    { steps { sh 'npx playwright test || true' } }
  }
  post {
    always {
      allure([results: [[path: 'allure-results']], reportBuildPolicy: 'ALWAYS']]
    }
  }
}

```

## 9.3 Docker

`docker/Dockerfile` extends the official Playwright image with Java 17, so one image can run tests *and* generate reports:

BASH

```

docker compose run --rm tests      # run tests + generate ./allure-report (history in ./allure-
history)
docker compose up report          # serve the report with nginx on http://localhost:8080

```

`--ipc=host` (set in `docker-compose.yml`) prevents Chromium from running out of shared memory.

## 9.4 Sharding and merging

With `npx playwright test --shard=1/4` on several machines, upload each machine's `allure-results/` as an artifact, download them all into one folder and generate a single report. Allure merges results by file, and every result file has a unique name. Set `ALLURE_KEEP_RESULTS=true` if shards share a workspace.

## 10. Command Reference

| Command                                                                                                                                        | Purpose                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>npm test</code>                                                                                                                          | Run all tests (headless, chromium)                                  |
| <code>npm run test:headed</code>                                                                                                               | Run with a visible browser                                          |
| <code>npm run test:debug</code>                                                                                                                | Playwright Inspector, one worker                                    |
| <code>npm run test:smoke</code> / <code>test:regression</code>                                                                                 | Tag-based subsets                                                   |
| <code>npm run test:auth</code> / <code>test:products</code> / <code>test:cart</code> / <code>test:checkout</code> / <code>test:showcase</code> | Suites                                                              |
| <code>npm run test:all-browsers</code>                                                                                                         | chromium, firefox, webkit, mobile                                   |
| <code>npm run test:staging</code>                                                                                                              | Use the staging profile                                             |
| <code>npm run test:report</code>                                                                                                               | Run tests, then always generate the report                          |
| <code>npm run allure:generate</code>                                                                                                           | Generate with history, metadata and theme                           |
| <code>npm run allure:single-file</code>                                                                                                        | One self-contained HTML report                                      |
| <code>npm run allure:open</code>                                                                                                               | Open the generated report                                           |
| <code>npm run allure:serve</code>                                                                                                              | Temporary report straight from results                              |
| <code>npm run trend:demo</code>                                                                                                                | Build several reports to populate trends                            |
| <code>npm run clean</code> / <code>clean:all</code>                                                                                            | Remove results/reports ( <code>clean:all</code> also drops history) |
| <code>npm run typecheck</code>                                                                                                                 | TypeScript type check                                               |
| <code>npm run docs:screenshots</code> / <code>docs:pdf</code>                                                                                  | Rebuild this tutorial's images / PDF                                |
| <code>SHOWCASE_FAILURES=false npm test</code>                                                                                                  | Skip the intentionally failing tests                                |
| <code>ALLURE_DETAIL=true npm test</code>                                                                                                       | Verbose Playwright steps in Allure                                  |
| <code>npx playwright show-trace &lt;trace.zip&gt;</code>                                                                                       | Open a trace downloaded from the report                             |

### Further reading

- Allure Report documentation: <https://allurereport.org/docs/>
- allure-playwright reference: <https://allurereport.org/docs/playwright-reference/>
- Playwright documentation: <https://playwright.dev/docs/intro>